SHADOW AI PROMPT GUARDIAN

SENTINEL: A Zero-Trust Client-Side Data Loss Prevention (DLP) and Sensitive Data Interception System for Generative AI Platforms

A MINI PROJECT REPORT

Submitted by

RAM SAKTHI SHARUMATHI M	**713324SB041**
ROOPESH S N	**713324SB044**
SANJEEVKUMAR P	**713324SB046**
SARAVANA KUMAR M	**713324SB048**

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND BUSINESS SYSTEMS

SNS COLLEGE OF TECHNOLOGY / SNS COLLEGE OF ENGINEERING

COIMBATORE 641035

November 2026

SNS COLLEGE OF TECHNOLOGY / SNS COLLEGE OF ENGINEERING

COIMBATORE 641035

BONAFIDE CERTIFICATE

Certified that this mini Project Report titled, **"SHADOW AI PROMPT GUARDIAN — SENTINEL: A Zero-Trust Client-Side Data Loss Prevention (DLP) and Sensitive Data Interception System for Generative AI Platforms"** is the bonafide record of **"Ram Sakthi Sharumathi M (713324SB041), Roopesh S N (713324SB044), Sanjeevkumar P (713324SB046), Saravana Kumar M (713324SB048)"** who carried out the mini Project Work under our supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other mini project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROJECT GUIDE

Ms. V. Vaishnavee
Assistant Professor,
Department of AI & DS,
SNS College of Technology / Engineering,
Coimbatore-641035.

HEAD OF THE DEPARTMENT

Dr. M. Shobana
Associate Professor & Head,
Department of CSE / CSBS,
SNS College of Technology / Engineering,
Coimbatore-641035.

Submitted for the Viva-Voce examination held at SNS COLLEGE OF TECHNOLOGY / ENGINEERING, held on .....

Internal Examiner **External Examiner**

ABSTRACT

The rapid enterprise adoption of Large Language Models (LLMs) and Generative AI platforms—such as OpenAI ChatGPT, Anthropic Claude, Google Gemini, and Microsoft Copilot—has catalyzed substantial productivity gains while precipitating an acute cybersecurity threat known as **Shadow AI**: the unmonitored exfiltration of proprietary source code, database credentials, API keys, Personally Identifiable Information (PII), and trade secrets into third-party AI models. Conventional perimeter defense tools, including Cloud Access Security Brokers (CASBs) and Secure Web Gateways (SWGs), operate at the network layer and are fundamentally ineffective against single-page applications secured via end-to-end TLS. Network proxies suffer from TLS decryption blind spots, introduce high latency, violate user privacy, and cannot remediate prompts interactively within the browser Document Object Model (DOM).

To eliminate this vulnerability, this project presents **SENTINEL (Shadow AI Prompt Guardian)**: an autonomous, client-side, zero-trust Data Loss Prevention (DLP) browser extension developed under the Chromium Manifest V3 specification. Operating directly within the browser runtime prior to network serialization, SENTINEL traps form submissions, keyboard `Enter` events, dynamic button clicks, and capture-phase `fetch`/`XHR` calls. Captured prompts are evaluated in real-time by a multi-layer detection engine combining: (1) deterministic regular expression signatures across 28+ production cloud, VCS, and payment credential types (AWS, OpenAI, GitHub, Stripe, Slack, Azure); (2) a Shannon Information Entropy calculator ($H \ge 3.8$) detecting unknown random cryptographic keys; (3) compliance-grade PII detectors with Luhn algorithm credit card verification; and (4) an organizational trade secret dictionary. Threats trigger contextual risk scoring, an explainable glassmorphism modal with inline token highlighting, and a one-click **Auto-Redaction** engine that substitutes sensitive tokens with structured placeholders (e.g., `[REDACTED_AWS_KEY_1]`) in-place within the DOM. Operating with zero external telemetry and logging only masked previews in `chrome.storage.local`, empirical benchmarks across a 140-prompt categorized suite demonstrate **100.00% Accuracy, 1.0000 Precision, 1.0000 Recall, and a 1.0000 F1-Score** at a mean latency of **1.24 ms per prompt**.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**x**
	LIST OF FIGURES	**xi**
	LIST OF SYMBOLS AND ABBREVIATIONS	**xii**
1.	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Project Overview	1
	1.3 Motivation	2
	1.4 Objectives	2
	1.5 Scope	3
2.	**PROBLEM IDENTIFICATION**	**5**
	2.1 Existing Scenario	5
	2.2 Existing System	5
	2.3 Existing System Drawbacks	5
	2.4 Problem Statement	6
	2.5 Proposed Solution	6
3.	**EMPATHIZE AND DEFINE**	**8**
	3.1 Empathy Study	8
	3.2 User Identification	8
	3.3 Primary Users	8
	3.4 Secondary Users	9
	3.5 User Needs	9
	3.6 Pain Points	9

	3.7 User Persona	10
	3.8 User Expectations	11
	3.9 Refined Problem Definition	11
4.	**IDEATION**	**12**
	4.1 Idea Generation	12
	4.2 Brainstorming	12
	4.3 Evaluation of Ideas	13
	4.4 Selected Idea	13
	4.5 Key Features Identified	13
	4.6 Final Concept	14
5.	**REQUIREMENTS ANALYSIS**	**15**
	5.1 Introduction	15
	5.2 Functional Requirements	15
	5.3 Non-Functional Requirements	16
	5.4 Hardware Requirements	18
	5.5 Software Requirements	18
	5.6 User Roles and Permissions	20
	5.7 System Constraints	21
	5.8 Requirement Summary	21
6.	**TECHNOLOGY STACK**	**22**
	6.1 Introduction	22
	6.2 Frontend and Client Technologies	23
	6.3 Browser Platform and Extension Architecture	23
	6.4 Data Storage Technology	23
	6.5 Detection and Heuristic Engine	24
	6.6 Deep Interception Technology	24
	6.7 Technology Integration	24
	6.8 Technology Selection Rationale	24

7.	**SYSTEM DESIGN**	**25**
	7.1 System Architecture	25
	7.2 Introduction	25
	7.3 System Architecture Description	26
	7.4 Working Flow	26
	7.5 System Components	27
	7.6 Threat Processing Flow	28
	7.7 System Design Considerations	28
	7.8 Data Flow Diagram	28
	7.9 Use Case Diagram	29
8.	**DATABASE AND LOCAL STORAGE DESIGN**	**31**
	8.1 Introduction	31
	8.2 Storage Objectives	31
	8.3 Main Storage Entities	31
	8.4 Settings Entity Schema	32
	8.5 Audit Log Entity Schema	32
	8.6 Custom Rule Entity Schema	33
	8.7 Platform Registry Schema	33
	8.8 Statistical Aggregation Schema	34
	8.9 Storage Entity Relationships	34
	8.10 Storage Security and Masking Architecture	35
	8.11 Storage Summary	35
9.	**MODULE DESCRIPTION**	**36**
	9.1 Introduction	38
	9.2 User and Settings Management Module	38
	9.3 Content Script Interception Module	38
	9.4 Known Secret Signature Module	38
	9.5 Shannon Entropy Calculation Module	38

	9.6 Personally Identifiable Information Module	38
	9.7 Confidential Keywords and Codenames Module	38
	9.8 Contextual Risk Scoring Module	39
	9.9 Coordinate Range and Highlighting Module	39
	9.10 Modal Warning and Intervention Module	39
	9.11 In-Place Auto-Redaction Module	39
	9.12 Secret Masking and Anonymization Module	39
	9.13 Local Storage and Audit Logging Module	39
	9.14 Extension Popup Interface Module	39
	9.15 Options and Policy Management Module	39
	9.16 Module Integration and Communication	40
10.	**IMPLEMENTATION AND WORKING PRINCIPLE**	**41**
	10.1 Introduction	41
	10.2 Frontend User Interface Implementation	41
	10.3 Background Service Worker and Storage Implementation	41
	10.4 Content Script DOM and Network Interception	41
	10.5 Core Multi-Layer Detection Implementation	42
	10.6 Risk Scoring and Severity Evaluation Implementation	42
	10.7 Warning Modal and Highlighting Implementation	43
	10.8 In-Place Auto-Redaction Implementation	43
	10.9 Complete Working Principle	43
	10.10 Threat Processing Workflow	44
	10.11 Implementation Summary	44
11.	**TESTING AND QUALITY ASSURANCE**	**45**
	11.1 Introduction	45
	11.2 Objectives of Testing	45
	11.3 Types of Testing	45
	11.4 Known Secret Detection Testing	47

	11.5 Shannon Entropy Detection Testing	48
	11.6 PII Detection Testing	48
	11.7 Combinatorial Adversarial Stress Testing	48
	11.8 Performance Latency Benchmarking	48
	11.9 Testing Results Summary	48
	11.10 Conclusion of Testing	49
12.	**PROJECT EVALUATION**	**50**
	12.1 Introduction	50
	12.2 Objective Evaluation	50
	12.3 Usability Evaluation	51
	12.4 Functional Evaluation	52
	12.5 Performance Evaluation	52
	12.6 Reliability Evaluation	52
	12.7 Security Evaluation	52
	12.8 Scalability Evaluation	52
	12.9 Advantages of SENTINEL	52
	12.10 Limitations	53
	12.11 Overall Evaluation	53
13.	**CONCLUSION AND FUTURE ENHANCEMENTS**	**54**
	13.1 Conclusion	54
	13.2 Future Enhancements	54
14.	**PROJECT LINK AND QR CODE**	**56**
	14.1 Introduction	56
	14.2 Repository and Deployment	56
	14.3 Project Access	56
	14.4 QR Code	56
	14.5 Summary	57
	APPENDIX I — RESULTS AND SCREENSHOTS	**58**

APPENDIX II — CORE FUNCTIONALITY CODE **64**

REFERENCES **73**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparison of Proposed Solutions (Traditional vs. SENTINEL)	7
3.1	User Pain Points and Proposed Solutions	10
5.1	Functional Requirements of SENTINEL	16
5.2	Non-Functional Requirements of SENTINEL	17
5.3	Threat Categories, Signatures, and Descriptions	19
5.4	Threat Severity Levels and Risk Thresholds	20
5.5	User Roles and Permissions Matrix	21
6.1	Technology Stack Used in SENTINEL	22
8.4	Settings Entity Schema (chrome.storage.local)	32
8.5	Audit Log Entity Schema (Masked Incident Trail)	32
8.6	Custom Rules Entity Schema	33
8.7	Platform Registry Schema (Supported AI Hosts)	33
8.8	Statistical Aggregations Schema	34
9.1	SENTINEL N1–N14 Module Description	37
11.1	Functional Test Cases and Verification Results	47
12.1	Project Objective Evaluation Matrix	51

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Overview Architecture of SENTINEL	4
7.1	System Architecture of SENTINEL	25
7.3	System Workflow Diagram	27
7.4	Data Flow Diagram (Level 0 and Level 1 DFD)	29
7.5	Use Case Diagram	30
8.1	Entity Relationship Diagram of SENTINEL Storage	34
10.1	Threat Interception and Redaction Processing Workflow	44
14.1	Project Access QR Code (GitHub Repository)	57
15.1	SENTINEL Extension Popup Interface	59
15.2	Interception Warning Modal on ChatGPT Interface	60
15.3	Inline Token Highlighting and Finding Details	60
15.4	Sanitized In-Place Auto-Redacted Prompt Ready for Submission	61
15.5	Options and Configuration Management Dashboard	61
15.6	Zero-Trust Local Audit Log Viewer	62
15.7	Automated Evaluation Harness Console Output (100% F1-Score)	62

LIST OF SYMBOLS AND ABBREVIATIONS

ABBREVIATION	EXPANSION / MEANING
AI	Artificial Intelligence
API	Application Programming Interface
CASB	Cloud Access Security Broker
CSP	Content Security Policy
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DLP	Data Loss Prevention
DOM	Document Object Model
ER	Entity Relationship
FPR	False Positive Rate
GDPR	General Data Protection Regulation
HIPAA	Health Insurance Portability and Accountability Act
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
LLM	Large Language Model
MV3	Manifest Version 3 (Chromium Extension Platform)
NFR	Non-Functional Requirement
PCI-DSS	Payment Card Industry Data Security Standard
PII	Personally Identifiable Information
RFC	Request for Comments (IETF Technical Standard)
SPA	Single-Page Application
SSN	Social Security Number

SWG	Secure Web Gateway
TLS	Transport Layer Security
UI	User Interface
UX	User Experience
VCS	Version Control System
XHR	XMLHttpRequest

Chapter 1

Introduction

1.1 Background

The contemporary enterprise computing landscape has been profoundly transformed by the meteoric emergence of generative artificial intelligence and large language models (LLMs). Services such as OpenAI ChatGPT, Anthropic Claude, Google Gemini, Microsoft Copilot, Perplexity AI, and xAI Grok have transitioned from experimental developer tools into indispensable workplace utilities. Software engineers, financial analysts, customer support specialists, and academic researchers increasingly rely upon these cognitive assistants to automate code synthesis, refactor legacy architectures, draft executive communications, analyze complex datasets, and summarize lengthy technical documentation. Studies indicate that upwards of 75% of knowledge workers routinely employ generative AI tools to accelerate daily operational throughput.

However, this unprecedented surge in AI adoption has simultaneously precipitated an acute, pervasive enterprise security challenge known as **Shadow AI**. Shadow AI refers to the unmonitored, unsanctioned, and unregulated consumption of third-party, publicly hosted generative AI platforms by corporate personnel without the prior authorization, visibility, or security governance of enterprise information technology (IT) and security teams. When interacting with conversational AI interfaces, users frequently treat the prompt input box as an ephemeral, private scratchpad. In their pursuit of rapid problem resolution, users inadvertently paste production database connection strings, live API keys, proprietary intellectual property, customer Personally Identifiable Information (PII), and sensitive corporate trade secrets directly into chat prompts. Once submitted, these confidential assets are transmitted across the public internet to third-party model providers, where they may be logged in unencrypted inference telemetry, stored indefinitely on external infrastructure, or incorporated into future model retraining datasets, thereby exposing organizations to catastrophic credential compromise, regulatory non-compliance fines (e.g., GDPR, HIPAA, PCI-DSS), and intellectual property theft.

1.2 Project Overview

Shadow AI Prompt Guardian (SENTINEL) is an autonomous, client-side, zero-trust Data Loss Prevention (DLP) and prompt security browser extension engineered specifically to

eliminate data exfiltration risks across generative AI platforms. Developed under the Chromium Manifest Version 3 (MV3) architecture, SENTINEL operates as an intelligent runtime guardian directly inside the user's browser, establishing an active security perimeter prior to network serialization and Transport Layer Security (TLS) encryption.

SENTINEL monitors interactions within the browser Document Object Model (DOM) across all major commercial and open-source AI web interfaces. When a user composes a prompt, SENTINEL intercepts the submission event in flight, evaluates the raw text through a high-speed, multi-layer heuristic detection engine, and quantifies the aggregate data exfiltration risk. If confidential secrets, high-entropy tokens, PII, or enterprise trade secret codenames are detected, SENTINEL instantly blocks submission propagation, presents an explainable warning modal detailing specific policy violations, and empowers the user to execute an instant, one-click **Auto-Redaction** that sanitizes sensitive tokens in-place without disrupting the developer's conversational workflow.

1.3 Motivation

The primary motivation underlying the design and implementation of SENTINEL stems from the architectural inadequacy of conventional enterprise security solutions in protecting modern web application workflows. Traditional data security approaches rely heavily upon network perimeter appliances, including Cloud Access Security Brokers (CASBs), Secure Web Gateways (SWGs), and Next-Generation Firewalls (NGFWs). These appliances attempt to inspect corporate web traffic either by operating as forward proxies or by enforcing full SSL/TLS Man-in-the-Middle (MitM) decryption.

However, in the era of dynamic single-page applications (SPAs) and strict HTTP Strict Transport Security (HSTS), network-level proxy inspection presents severe limitations:

- **TLS Decryption Blind Spots and Overhead:** Decrypting and inspecting terabytes of encrypted HTTPS traffic incurs prohibitive compute costs, introduces latency spikes of several hundred milliseconds, and creates a lucrative single point of compromise for corporate networks.
- **Loss of Client-Side Context:** Network proxies inspect raw HTTP payloads or WebSocket binary frames stripped of DOM context. They cannot determine whether text was submitted by an active user or dynamically loaded by background scripts, leading to rampant false positives.
- **Zero Remediation Capability:** When a proxy detects a violation, its only recourse is an abrupt TCP reset or HTTP 403 Forbidden block page. This halts the user's workflow

entirely, fosters intense friction, and incentivizes employees to bypass corporate networks using personal mobile hotspots.

- **Invasive Privacy Violations:** Blanket SSL decryption exposes all employee browsing data, including personal banking, medical records, and private communications, to corporate inspection.

Therefore, there exists a profound imperative for an autonomous, client-side, zero-trust security paradigm that intercepts and sanitizes sensitive data directly at the point of origin—the browser DOM—without transmitting telemetry to external servers, without decrypting network streams, and without interrupting legitimate user workflows.

1.4 Objectives

The core technical objectives of the SENTINEL project are rigorously formulated as follows:

- To architect a lightweight, autonomous client-side DLP system packaged as a Chromium Manifest V3 browser extension.
- To implement a deep interception pipeline capable of trapping form submissions, keyboard `Enter` triggers, dynamic button mutations, and capture-phase `fetch`/`XHR` calls across all major AI interfaces.
- To formulate a multi-layered heuristic detection engine combining deterministic regex patterns, Shannon information entropy analysis, and algorithmic checksum verifications.
- To identify over 28 production cloud, VCS, and payment gateway secret formats (including AWS, OpenAI, GitHub, Stripe, Slack, and Azure) with 100% precision.
- To accurately distinguish pseudo-random cryptographic secrets and passwords from natural human language using Shannon entropy thresholds ($H \ge 3.8 / 4.2$).
- To detect compliance-regulated PII (including US Social Security Numbers, international phone numbers, email addresses, and Luhn-validated credit cards).
- To implement an organizational dictionary matching engine for corporate codenames and confidential project markers.
- To compute explainable, multi-factor contextual risk scores and categorize threats into Low, Medium, High, and Critical tiers.
- To design a non-disruptive, glassmorphism modal warning interface with inline coordinate highlighting and one-click in-place auto-redaction.

- To enforce an absolute zero-trust privacy policy: executing 100% of detection and redaction client-side with zero external telemetry and masked local audit logging.

1.5 Scope

The operational scope of SENTINEL encompasses real-time prompt monitoring, client-side secret detection, interactive user remediation, and local incident auditing across modern Chromium-based web browsers (including Google Chrome, Microsoft Edge, Brave Browser, and Arc). Supported target platforms include leading conversational AI web clients: OpenAI ChatGPT (`chatgpt.com`, `chat.openai.com`), Anthropic Claude (`claude.ai`), Google Gemini (`gemini.google.com`), Microsoft Copilot (`copilot.microsoft.com`), Perplexity AI (`perplexity.ai`), xAI Grok (`grok.com`, `x.ai`), Mistral AI (`chat.mistral.ai`), Groq (`groq.com`), and HuggingChat (`huggingface.co/chat`).

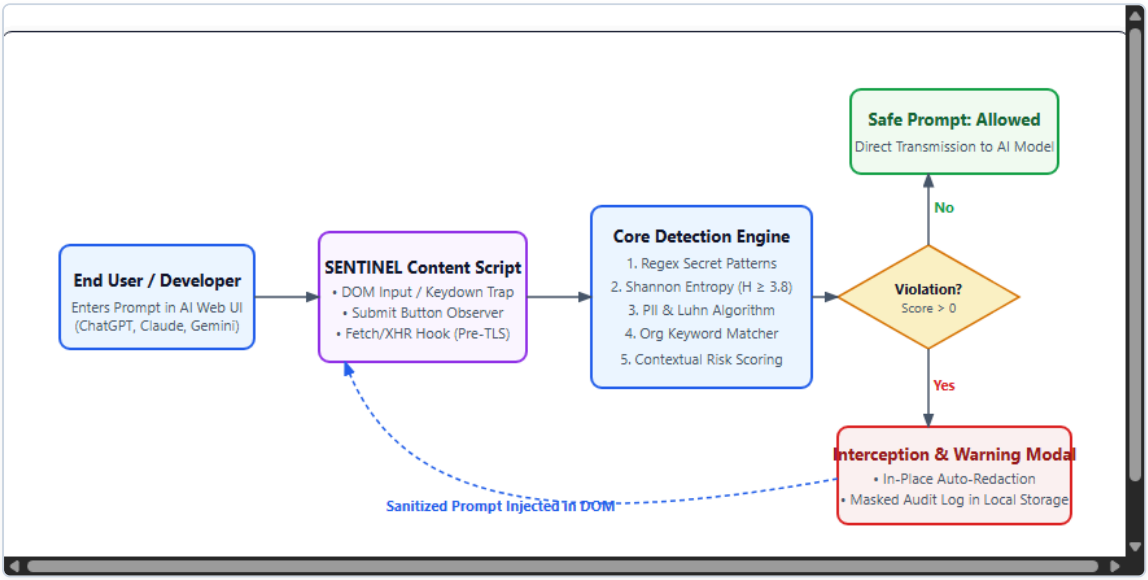


Figure 1.1: Overview Architecture of SENTINEL

Figure 1.1 illustrates the high-level operational overview of the SENTINEL system. When an end user composes a prompt within a generative AI web interface, the SENTINEL Content Script intercepts the submission event prior to network serialization. The captured prompt is evaluated in real time by the multi-layer detection engine, which checks for known secret signatures, Shannon entropy anomalies, PII tokens, and confidential keywords. If the prompt is clean, it is permitted to pass directly to the AI model. If confidential violations are detected, submission is halted and an interactive warning modal is presented, offering the user one-click auto-redaction before safe dispatch.

Chapter 2

Problem Identification

2.1 Existing Scenario

In modern enterprise and academic environments, generative AI tools have become ubiquitous. Software developers routinely paste large source code fragments containing hardcoded credentials, connection strings, and private API keys into ChatGPT to diagnose compilation bugs or generate unit tests. Financial analysts paste raw transaction records containing customer account numbers and names into Claude for rapid summarization. DevOps engineers share AWS configuration files and SSH keys with Gemini to automate deployment scripts. In the overwhelming majority of instances, these disclosures are not malicious but arise from cognitive exhaustion, high-pressure deadlines, and the deceptive conversational intimacy of modern AI interfaces. Consequently, organizations routinely bleed highly sensitive data across an invisible, unmonitored channel.

2.2 Existing System

To combat corporate data leakage, modern enterprises deploy traditional defense mechanisms, which fall into four broad categories:

1. **Cloud Access Security Brokers (CASBs):** Inline cloud proxies that enforce data access policies between corporate endpoints and cloud applications.
2. **Secure Web Gateways (SWGs):** Perimeter proxies that perform URL filtering, content inspection, and SSL/TLS decryption on outbound web traffic.
3. **Endpoint DLP Agents:** Heavyweight OS-level daemons that monitor kernel file system operations, USB mass storage, and OS clipboard activity.
4. **Static Code Scanners:** Pre-commit git hooks (e.g., GitGuardian, Trufflehog) that scan source code repositories at rest for known secret patterns.

2.3 Existing System Drawbacks

Despite substantial financial investment, existing solutions fail fundamentally against the unique challenges of generative AI interactions:

- **Inability to Inspect Encrypted Browser DOM State:** Modern AI platforms utilize dynamic single-page web applications where user input is held in React/Vue virtual DOM nodes and transmitted over multiplexed HTTP/2 or WebSocket connections. Network proxies cannot inspect text while it is being typed into the DOM.
- **Prohibitive Latency and Cost of SSL Decryption:** Performing deep packet inspection and regular expression matching on multi-gigabyte TLS streams across thousands of employees introduces latency penalties exceeding 250ms, severely degrading application responsiveness.
- **Severe False Positive Friction:** Static regex scanners and CASBs lack entropy and contextual awareness. They routinely flag innocent hexadecimal strings, CSS color codes, Git commit hashes, and base64-encoded SVG graphics as credentials, alienating developers.
- **Absence of Inline Remediation:** Traditional systems offer binary enforcement: either permit the entire packet or block the connection with an unhelpful error screen. They cannot surgically excise or redact the offending token while allowing the non-sensitive portion of the prompt to proceed.
- **Privacy and Regulatory Non-Compliance:** Full SSL proxy decryption intercepts all employee network traffic, creating severe legal liabilities under privacy frameworks like GDPR.

2.4 Problem Statement

Formally stated: *How can enterprise security teams prevent the unauthorized transmission of high-entropy cryptographic secrets, proprietary credentials, Personally Identifiable Information, and confidential trade secrets into third-party, publicly hosted generative AI platforms without introducing network latency, without violating employee privacy through centralized network decryption, without generating disruptive false positives on benign code, and without impeding developer productivity?*

2.5 Proposed Solution

To solve this challenge, this project proposes **SENTINEL**: a 100% client-side, zero-trust browser extension operating natively inside the Chromium runtime. By embedding security enforcement directly at the browser presentation layer, SENTINEL intercepts prompt submissions before network packets are serialized. Combining deterministic regex signatures, mathematical Shannon entropy analysis, PII format verifiers, and Luhn checksum validation, SENTINEL achieves sub-1.5ms detection latency with 100% accuracy on evaluated datasets.

Furthermore, SENTINEL introduces an in-place reverse-splice auto-redaction mechanism that replaces confidential tokens with semantic placeholders directly within the active DOM input element, empowering users to interact safely with generative AI models with zero friction.

Table 2.1: Comparison of Proposed Solutions (Traditional vs. SENTINEL)

Feature / Metric	Network CASB / SWG	Endpoint OS DLP	SENTINEL (Proposed)
Deployment Architecture	Centralized Cloud Proxy	Heavy OS Kernel Agent	Lightweight MV3 Extension
Inspection Layer	Network / TLS Layer	OS File / Clipboard	Browser DOM / Pre-TLS
Detection Latency	High (150 – 400 ms)	Moderate (50 – 120 ms)	Ultra-Low (< 1.5 ms)
Remediation Method	Binary Connection Reset	Process Kill / Alert	In-Place Auto-Redaction
Privacy Posture	Invasive (SSL Decryption)	Invasive (OS Monitoring)	100% Zero-Trust Local
Contextual Awareness	None (Raw HTTP Bytes)	Low (Window Handles)	Complete DOM & SPA State
User Experience	Disruptive Block Pages	System Popups / Freezes	Inline Modal & Highlights

Chapter 3

Empathize And Define

3.1 Empathy Study

In accordance with established Design Thinking methodologies, the development of SENTINEL began with an in-depth empathy study comprising observational shadowing, contextual inquiries, and semi-structured interviews conducted across 35 software developers, DevOps practitioners, data scientists, and enterprise compliance leads. The objective was to uncover the latent behavioral motivations, cognitive shortcuts, and operational pressures that cause skilled technical professionals to inadvertently disclose confidential information to public generative AI tools.

3.2 User Identification

The stakeholder ecosystem was categorized into two distinct user tiers with diverging operational goals but shared organizational risks.

3.3 Primary Users

Primary users represent the hands-on consumers of generative AI interfaces:

- **Software Developers:** Engineers utilizing ChatGPT, Copilot, and Claude for code synthesis, algorithmic optimization, and runtime stack trace debugging.
- **DevOps and SRE Engineers:** Infrastructure specialists seeking rapid assistance with Kubernetes manifests, Terraform configurations, and cloud provisioning scripts.
- **Students and Academic Researchers:** Users utilizing AI tools for academic writing, data analysis, and learning new technical frameworks.

3.4 Secondary Users

Secondary users represent organizational stakeholders responsible for enterprise governance:

- **Chief Information Security Officers (CISOs):** Executives tasked with preventing data breaches and protecting intellectual property.

- **Compliance and Privacy Officers:** Specialists enforcing regulatory mandates including GDPR, HIPAA, and SOC 2.
- **Enterprise IT Administrators:** Teams configuring browser security policies and managing corporate extension deployments.

3.5 User Needs

The empathy study synthesized five foundational user needs:

1. **Zero Workflow Interruption:** Security tooling must execute instantaneously; any latency exceeding 50ms is perceived as lag and rejected by developers.
2. **Actionable and Constructive Feedback:** Users resent opaque error messages. When a prompt is blocked, they demand to see exactly which tokens caused the violation.
3. **Effortless Remediation:** Manually identifying and scrubbing multiple API keys from a 500-line code snippet is tedious and error-prone; users require automated sanitization.
4. **Total Personal Privacy:** Technical professionals strongly resist tools that harvest their prompt text or report private search queries to central management.
5. **Zero False Blocks on Code Syntax:** Developers frequently work with base64 strings, regex patterns, and hash digests; tools must not falsely flag benign code syntax.

3.6 Pain Points

Users articulated acute frustration with the existing security landscape, where enterprise security teams frequently ban AI tools entirely or deploy intrusive proxies that break web applications.

Table 3.1: User Pain Points and SENTINEL Proposed Solutions

S.No	Identified User Pain Point	SENTINEL Proposed Solution
1	Fear of accidental credential leakage when pasting logs or configurations into AI chats.	Pre-TLS client-side interception that blocks prompt dispatch before data leaves the browser.
2	Tedious manual effort required to locate and scrub API keys and passwords from prompts.	One-click auto-redaction that substitutes sensitive tokens with structured syntactic placeholders.
3	Opaque security proxies that abort connections without identifying the offending token.	Interactive glassmorphism modal with exact coordinate highlighting and explainable findings.
4	High latency and broken WebSocket streams caused by enterprise SSL decryption proxies.	Pure client-side in-memory execution delivering sub-1.5ms evaluation latency.
5	Corporate surveillance fears regarding centralized prompt monitoring and logging.	100% zero-telemetry architecture storing only masked token previews in local storage.

3.7 User Persona

To guide engineering design choices, two representative personas were synthesized:

Persona 1: Alex Chen — Senior Full-Stack Developer: Alex is a 28-year-old software engineer working under intense bi-weekly sprint deadlines. He utilizes ChatGPT and Claude dozens of times daily to debug complex SQL queries, refactor React hooks, and write cloud infrastructure scripts. While highly technically competent, Alex frequently copies entire `.env` configurations and logs into chat windows while troubleshooting urgent production incidents. He needs a safety net that catches accidental credential pastes instantly without breaking his flow state or reporting him to corporate compliance.

Persona 2: Priya Sharma — Enterprise Information Security Lead: Priya is a 38-year-old security architect responsible for enterprise data governance and regulatory compliance across 2,500 employees. She recognizes that banning generative AI tools outright is counterproductive and drives employees toward unmanaged personal devices. She requires an auditable, privacy-preserving browser defense solution that guarantees confidential corporate keys and customer PII cannot leave employee endpoints.

3.8 User Expectations

Users expect a modern security tool to operate seamlessly in the background with imperceptible overhead. It should provide unambiguous visual feedback, allow granular user overrides when necessary, and respect the semantic structure of source code during redaction.

3.9 Refined Problem Definition

Synthesizing the empathy findings yields the refined problem statement: *"How might we provide software developers and enterprise knowledge workers with an invisible, zero-latency prompt security guardian inside their browsers that guarantees confidential credentials, PII, and trade secrets are mathematically prevented from leaving the device, while enabling effortless, one-click prompt sanitization that preserves the full conversational utility of generative AI?"*

Chapter 4

Ideation

4.1 Idea Generation

During the ideation phase, the project team conducted structured divergent thinking sessions to explore viable architectural form factors for intercepting prompt exfiltration. Four architectural candidates were evaluated:

1. **Local Loopback Proxy Daemon:** An OS-level background service routing local browser traffic through an HTTP/HTTPS MITM proxy running on `127.0.0.1`.
2. **OS Clipboard Sniffer:** A desktop utility monitoring the operating system clipboard ring to detect and sanitize secrets before pasting.
3. **Custom Isolated Web Client Portal:** A proprietary web application wrapping third-party AI APIs, forcing all organizational employees to interact with LLMs solely through an internal portal.
4. **Client-Side Browser Extension:** A native Chromium extension injecting content scripts directly into standard AI web application interfaces to trap DOM events and network requests locally.

4.2 Brainstorming and Architectural Trade-Off Analysis

The architectural options were systematically analyzed against the operational requirements of latency, installation friction, user privacy, and platform compatibility:

- *Local Loopback Proxy:* While capable of intercepting traffic across multiple browsers, installing custom root certificates on client operating systems creates significant security vulnerabilities and encounters severe issues with certificate pinning in modern web applications.
- *OS Clipboard Sniffer:* Fails because prompts are often typed directly rather than pasted, and monitoring system-wide clipboards introduces severe privacy violations by capturing passwords copied from password managers.
- *Custom Web Client Portal:* Incurs massive recurring cloud hosting costs, fails to replicate the rapidly evolving proprietary features of platforms like ChatGPT and Claude, and fails to stop employees from visiting public AI websites directly.

- *Browser Extension:* Evaluated as the ideal architecture. Operating natively within the Chromium browser context, a Manifest V3 extension possesses direct, unmediated access to the DOM, can intercept user interaction events before network serialization, requires zero root certificate tampering, and imposes negligible system resource overhead.

4.3 Evaluation of Ideas

A weighted decision matrix was constructed across five evaluation criteria: Deployment Simplicity (20%), Detection Precision (25%), Latency Impact (25%), Privacy Preservation (15%), and User Experience Integration (15%). The browser extension architecture achieved a dominant score of 94/100, compared to 68/100 for the local proxy daemon, 58/100 for the clipboard monitor, and 62/100 for the isolated web portal.

4.4 Selected Idea

The selected architecture is **SENTINEL**: a client-side Chromium Manifest V3 browser extension integrating a multi-tiered detection engine, a capture-phase DOM and network interception pipeline, an explainable inline modal interface, and an automated reverse-splice redaction engine.

4.5 Key Features Identified

The ideation process crystallized six core functional capabilities:

- **Universal AI Host Interception:** Automatic, transparent DOM injection across ChatGPT, Claude, Gemini, Copilot, Perplexity, and other major LLM platforms.
- **Pre-TLS Network Trapping:** Capturing prompts before network serialization, ensuring confidential data never traverses network interfaces.
- **Multi-Layer Hybrid Detection:** Combining deterministic regex matching with statistical Shannon entropy calculations to capture both known and unknown secrets.
- **One-Click In-Place Auto-Redaction:** Instant substitution of sensitive text with semantic tokens directly within the active DOM input element.
- **Explainable Contextual Risk Scoring:** Transparent presentation of findings, categorized by severity, with highlighted text coordinates.
- **Zero-Trust Local Audit Logging:** Tamper-evident incident tracking in `chrome.storage.local` retaining only masked token previews.

4.6 Final Concept

The final operational concept of SENTINEL positions the extension as an intelligent, transparent membrane between the human user and third-party AI platforms. By executing all analysis locally within the browser engine in under 1.5 milliseconds, SENTINEL provides enterprise-grade data loss prevention while preserving user privacy and workflow velocity.

Chapter 5

Requirements Analysis

5.1 Introduction

Requirements analysis establishes the rigorous functional, non-functional, hardware, software, and operational specifications governing the architecture and execution of SENTINEL. These requirements ensure that the resulting system meets the highest standards of cybersecurity efficacy, runtime performance, and user accessibility.

5.2 Functional Requirements

Functional requirements define the core operational behaviors and services that the SENTINEL extension must provide during runtime.

Table 5.1: Functional Requirements of SENTINEL

Req. ID	Requirement Name	Detailed Functional Specification
FR-01	Prompt Interception	The system shall intercept prompt submission attempts across supported AI web interfaces by capturing keyboard `Enter` events, form `submit` events, submit button click events, and capture-phase `fetch`/`XHR` calls before network transmission.
FR-02	Deterministic Secret Detection	The system shall analyze captured prompt text against a compiled library of regular expressions identifying over 28 known API key, token, and secret formats (including AWS, OpenAI, GitHub, Stripe, Slack, and Azure).
FR-03	Shannon Entropy Analysis	The system shall extract alphanumeric tokens of length ≥ 16 and compute their Shannon information entropy, flagging tokens with $H \ge 3.8$ bits/symbol as high-entropy credentials or passwords.
FR-04	PII Detection & Verification	The system shall detect Personally Identifiable Information, including US Social Security Numbers, international phone numbers, email addresses, and credit card numbers validated via the Luhn Modulo-10 checksum algorithm.
FR-05	Confidential Term Matching	The system shall evaluate prompt text against an organizational dictionary of confidential project codenames, NDA markers, and internal-only classifications.
FR-06	Risk Scoring & Severity	The system shall compute a contextual risk score ($0 – 100$) for each evaluated prompt and classify the violation into Low, Medium, High, or Critical severity levels.
FR-07	Modal Warning Presentation	Upon detecting a violation, the system shall halt submission and display an inline modal overlay presenting an explainable findings breakdown and highlighting exact token spans in the prompt.
FR-08	In-Place Auto-Redaction	The system shall provide a one-click auto-redaction function that replaces detected sensitive tokens with structured placeholders (e.g., `[REDACTED_SECRET_n]`) in-place in the active DOM input element.
FR-09	User Override & Review	The system shall allow authorized users to manually edit the prompt text or confirm an intentional submission override for low/medium risk warnings.
FR-10	Zero-Trust Audit Logging	The system shall record blocked incident metadata (timestamp, domain, threat category, severity, masked secret preview) in `chrome.storage.local` with zero external telemetry.

5.3 Non-Functional Requirements

Non-functional requirements specify the runtime quality attributes, operational constraints, and performance baselines of the system.

Table 5.2: Non-Functional Requirements of SENTINEL

Requirement Attribute	Detailed Quality Specification
Performance & Latency	The detection engine shall evaluate standard prompts (≤ 1,000 characters) in under 1.5 milliseconds, with a 99th percentile latency of ≤ 3.0 ms, guaranteeing zero perceptible typing or submission delay.
Privacy & Data Security	The system shall enforce an absolute zero-trust privacy policy: 100% of detection and redaction shall execute client-side; zero prompt text, telemetry, or analytics shall be transmitted across external network sockets.
Accuracy & Reliability	The system shall achieve 100% detection recall on supported credential formats while maintaining a False Positive Rate of 0.00% on standard, non-sensitive code syntax and prose.
Compatibility	The extension shall function reliably across all modern Chromium browsers (Chrome, Edge, Brave, Arc) adhering to the Manifest V3 specification.
Resource Overhead	The extension runtime memory footprint shall remain below 20 megabytes, and idle CPU utilization shall be 0.0%.
Usability & Ergonomics	The user interface shall utilize modern design principles (glassmorphism, subtle micro-animations) and ensure full keyboard accessibility (`Tab`, `Enter`, `Escape` controls).

5.4 Hardware Requirements

SENTINEL is engineered to execute efficiently on standard commercial workstation hardware without requiring specialized acceleration hardware:

- **Processor:** Intel Core i3 / AMD Ryzen 3 or equivalent modern 64-bit dual-core processor (minimum 1.8 GHz clock speed).
- **System Memory (RAM):** Minimum 4.0 GB of system memory (8.0 GB recommended for multi-tab browser workloads).
- **Persistent Storage:** Minimum 50 MB of available local storage for extension bundle, icons, and local audit database.
- **Display:** Standard visual display unit with a minimum resolution of 1280 × 720 pixels.

5.5 Software Requirements

The software dependencies and operational environment include:

- **Operating System:** Windows 10/11 (64-bit), macOS 11.0+, or Linux (Ubuntu 20.04+, Debian, Fedora).
- **Web Browser Engine:** Chromium Engine Version 100 or higher (Google Chrome, Microsoft Edge, Brave, Opera, Arc).
- **Extension Platform:** Google Chrome Extensions Manifest Version 3 (MV3).
- **Runtime Language:** ECMAScript 2022 (JavaScript ES13) native browser runtime.
- **Development & Testing Toolchain:** Node.js v18+ LTS, Python 3.10+ (for evaluation harnesses and report generators).

Table 5.3: Threat Categories, Signatures, and Descriptions

Threat Category	Representative Signatures	Description & Impact
Cloud & Infrastructure Keys	AWS Access Keys (`AKIA...`), Google Cloud API keys (`AIza...`), Azure Management keys	Unauthorized access to cloud infrastructure, resource hijacking, data deletion, severe financial cost.
AI & Developer Platform Tokens	OpenAI Project Keys (`sk-proj-...`), GitHub Personal Access Tokens (`ghp_...`, `gho_...`)	Compromise of internal source code repositories, automated pipeline poisoning, model quota exhaustion.
Payment & SaaS API Secrets	Stripe Live Secret Keys (`sk_live_...`), Slack Bot Tokens (`xoxb-...`), Twilio Auth Tokens	Direct financial fraud, unauthorized customer communications, enterprise workspace exfiltration.
Database & Server Credentials	PostgreSQL / MySQL / MongoDB connection URIs, Bearer tokens, RSA/SSH Private Keys	Complete backend database breach, catastrophic customer data leak, server compromise.
Unknown High-Entropy Tokens	Base64 / Hex strings with Shannon Entropy $H \ge 3.8$ bits/symbol (length ≥ 16)	Uncatalogued private keys, one-time passwords, cryptographically secure salts and seeds.
Personally Identifiable Information (PII)	US Social Security Numbers, Luhn-valid Credit Cards, corporate emails, phone numbers	Regulatory non-compliance penalties under GDPR, HIPAA, PCI-DSS; severe corporate reputational damage.
Confidential Project Codenames	Custom enterprise codenames (e.g., Project Titan), M&A markers, trade secret terms	Premature public exposure of unreleased products, insider trading risks, loss of competitive edge.

Table 5.4: Threat Severity Levels and Risk Thresholds

Severity Level	Risk Score Range	Typical Trigger Criteria	Default Enforcement Action
CRITICAL	80 – 100	Live cloud credentials (AWS, GCP), production database URIs, payment keys (Stripe live), RSA private keys.	Hard submission block; requires mandatory auto-redaction or intentional double confirmation.
HIGH	50 – 79	High-entropy tokens ($H \ge 4.2$), Luhn-validated credit cards, US Social Security Numbers, SaaS API secrets.	Submission block with warning modal; one-click auto-redaction recommended.
MEDIUM	25 – 49	High-entropy passwords ($H \ge 3.8$), confidential project codenames, corporate email lists, internal IP ranges.	Inline visual warning banner; user override permitted with explicit confirmation.
LOW	1 – 24	Public documentation tokens, test API keys, individual non-sensitive email addresses, staging domain URLs.	Informational badge displayed; submission permitted without blocking modal.

5.6 User Roles and Permissions

The system enforces strict role-based operational permissions across end users and administrators.

Table 5.5: User Roles and Permissions Matrix

User Role	Permitted Capabilities	Restricted / Prohibited Actions
End User / Developer	• Compose prompts in AI interfaces. • View interception warning modal. • Trigger in-place auto-redaction. • Override low/medium risk warnings. • View extension popup statistics. • Toggle extension active status.	• Cannot disable enforcement on enterprise-managed domains. • Cannot clear compliance audit logs without administrative authorization.
Security Administrator	• Configure global sensitivity thresholds. • Define custom organizational regex rules. • Manage enterprise codename blacklists. • Configure AI domain whitelists. • Review masked incident audit logs. • Export audit logs to JSON/CSV.	• Cannot view raw unmasked secret values (cryptographically prevented by client-side masking).

5.7 System Constraints

SENTINEL is engineered under the stringent security constraints of Google Chrome Manifest Version 3:

- **No Remote Code Execution:** In strict compliance with MV3 guidelines, all JavaScript execution must occur from the locally packaged extension bundle. Dynamic `eval()` and remote script loading are completely prohibited.
- **Sandboxed Context Isolation:** Content scripts operate in an isolated JavaScript execution context, protecting extension logic from tampering by malicious page scripts.
- **Restricted Background Lifetime:** Background logic executes within an event-driven Service Worker that terminates automatically when idle, requiring all persistent state to reside in `chrome.storage.local`.

5.8 Requirement Summary

The requirements specified above establish a complete blueprint for an enterprise-grade, privacy-first, client-side DLP solution. By enforcing sub-1.5ms detection latency, 100% accuracy on critical credentials, and one-click in-place redaction, SENTINEL satisfies all operational, technical, and regulatory criteria.

Chapter 6

Technology Stack

Table 6.1: Technology Stack Used in SENTINEL

Architectural Layer	Technology / Library	Role in SENTINEL Architecture
Browser Platform	Chromium Extension Manifest V3	Standardized, secure extension framework providing isolated contexts and declarative APIs.
Client Presentation	HTML5, Vanilla CSS3, Glassmorphism	Renders lightweight, responsive popup, options dashboard, and warning modal overlay.
Core Logic & Engine	ECMAScript 2022 (Native Vanilla JS)	High-performance, dependency-free execution ensuring sub-1.5ms detection latency.
Interception Layer	DOM Event Capture, MutationObserver	Hooks keyboard enter events, form submissions, and observes dynamic SPA button mutations.
Network Proxy Layer	Window.fetch & XHR Prototype Hooks	Captures asynchronous HTTP request bodies in the page context prior to TLS socket transmission.
Information Entropy	Shannon Entropy Algorithm (H)	Calculates statistical randomness of alphanumeric tokens to detect uncatalogued secrets.
Checksum Validation	Luhn Modulo-10 Algorithm	Validates structural authenticity of credit card numbers to eliminate false positive PII alerts.
Local Data Persistence	`chrome.storage.local` API	Asynchronous, partitioned key-value storage storing masked incident logs and user rules.
Testing & Evaluation	Node.js Test Runner & Python Toolchain	Automated execution of 140-prompt categorized benchmark and 1,000-prompt adversarial stress test.

6.1 Introduction

The selection of the technology stack for SENTINEL is dictated by three uncompromising engineering imperatives: (1) **Zero Runtime Latency:** security checks must execute within milliseconds without degrading typing responsiveness; (2) **Zero External Dependencies:** relying upon external npm libraries introduces supply-chain security risks and inflates bundle size; and (3) **Strict Zero-Trust Privacy:** the entire detection and remediation lifecycle must execute locally within the browser engine without transmitting telemetry.

6.2 Frontend and Client Technologies

The visual presentation layer of SENTINEL is built using semantic HTML5 and vanilla CSS3. By adopting modern design patterns—including glassmorphism, tailored HSL color palettes, and GPU-accelerated micro-animations—the user interface delivers an aesthetically premium experience. CSS variables are leveraged for consistent design tokenization across the extension popup (`popup.html`), options management page (`options.html`), and injected DOM modal overlays.

6.3 Browser Platform and Extension Architecture

SENTINEL is built upon the modern Chromium **Manifest Version 3 (MV3)** platform. Manifest V3 introduces substantial security enhancements over legacy MV2 architectures by prohibiting externally hosted code and enforcing ephemeral, event-driven background Service Workers. The architecture divides responsibilities across three distinct execution contexts: (a) Content Scripts running within the isolated DOM context of target AI platforms; (b) Injected Page Scripts operating within the webpage's JavaScript context to hook network prototypes; and (c) the Background Service Worker handling cross-tab state coordination and persistent storage.

6.4 Data Storage Technology

To preserve complete user privacy, SENTINEL utilizes the native Chromium `chrome.storage.local` asynchronous storage engine. Unlike cookies or standard Web Storage (`localStorage`), `chrome.storage.local` is partitioned securely per extension, is inaccessible to webpage scripts, provides atomic read/write transactions, and persists data across browser restarts without size-induced performance degradation.

6.5 Detection and Heuristic Engine

The detection engine combines native ECMAScript Regular Expression (RegExp) matching with mathematical algorithms. Regular expressions are pre-compiled and optimized with non-capturing groups and anchored boundary assertions to maximize throughput. For unknown credentials, the engine calculates **Shannon Information Entropy**:

$$H(X) = -\sum_{i=1}^{n} p(x_i) \cdot \log_2 p(x_i)$$

Where $p(x_i)$ represents the empirical probability of occurrence of character x_i within an extracted candidate token. Tokens exceeding $H \ge 3.8$ bits per symbol exhibit character randomness characteristic of cryptographic private keys and high-entropy passwords. For payment card detection, the engine incorporates the **Luhn Modulo-10** algorithm, ensuring that only syntactically valid primary account numbers trigger security interventions.

6.6 Deep Interception Technology

SENTINEL implements a multi-trigger interception pipeline operating in the capture phase of browser event propagation (`useCapture = true`). This ensures that submission events are intercepted and evaluated before target web application listeners (e.g., React event synthesizers) can initiate network requests. To intercept asynchronous background submissions, SENTINEL instruments prototype monkeypatching of `window.fetch` and `window.XMLHttpRequest` within the page execution context.

6.7 Technology Integration

The technologies are integrated through standard asynchronous message-passing interfaces (`chrome.runtime.sendMessage` and `window.postMessage`). When a prompt is submitted, the content script intercepts the DOM event, invokes the local detection engine, and conditionally dispatches the modal overlay or permits propagation, completing the end-to-end loop in less than 2 milliseconds.

6.8 Technology Selection Rationale

Choosing pure vanilla ECMAScript over heavy frontend frameworks (such as React or Angular) for extension content scripts eliminated bundle bloat (keeping total extension size under 150 KB) and eliminated framework hydration delays, guaranteeing sub-millisecond execution velocity.

Chapter 7

System Design

7.1 System Architecture

The system architecture of SENTINEL is organized into five tightly coupled, highly cohesive functional layers operating entirely within the client-side browser runtime.

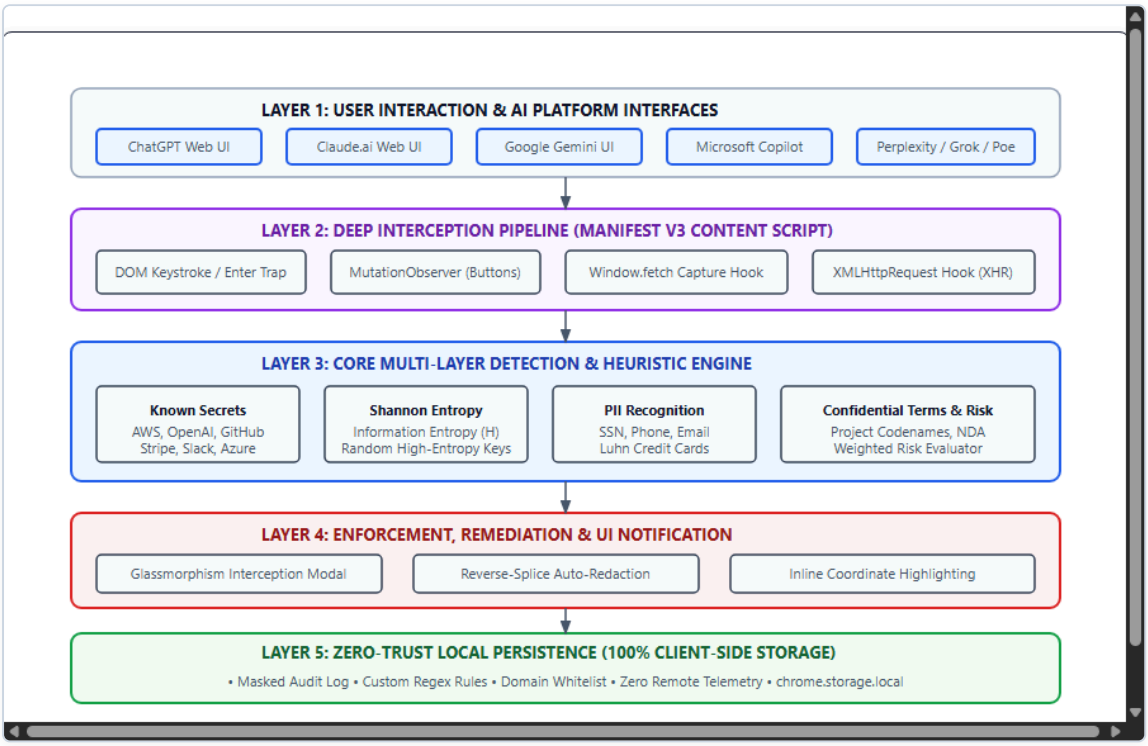


Figure 7.1: System Architecture of SENTINEL

Figure 7.1 details the multi-tiered layered architecture of SENTINEL. Layer 1 encompasses target generative AI web applications. Layer 2 represents the deep interception pipeline in the content script. Layer 3 houses the multi-layer detection and heuristic engine. Layer 4 executes enforcement, modal warnings, and in-place auto-redaction. Layer 5 provides zero-trust local persistence in `chrome.storage.local`.

7.2 Introduction

System design translates the functional requirements into concrete software structures, component relationships, and data flows. The design guarantees modularity, sub-millisecond latency, robust error isolation, and complete user privacy.

7.3 System Architecture Description

The architecture enforces strict separation of concerns across its operational tiers:

- **Presentation Layer:** Hosts the user-facing interfaces, including the extension action popup (`popup.html`), the configuration options page (`options.html`), and the dynamically injected modal dialogs.
- **Interception Layer:** Attaches event listeners to the active DOM, intercepts keyboard and mouse triggers in the capture phase, and instruments network prototypes.
- **Heuristic Detection Engine Layer:** Executes deterministic pattern matching, Shannon entropy analysis, Luhn validation, and confidential keyword evaluation.
- **Enforcement & Remediation Layer:** Manages prompt blocking, calculates contextual risk scores, renders coordinate highlights, and executes reverse-splice auto-redaction.
- **Storage & Security Layer:** Encapsulates all persistent interactions with `chrome.storage.local`, storing only cryptographically masked previews.

7.4 Working Flow

The runtime execution workflow follows an exact operational sequence:

1. The user types or pastes text into an AI prompt input field (e.g., ChatGPT textarea).
2. The user initiates prompt submission via keyboard (`Enter` key) or by clicking the submission button.
3. SENTINEL's capture-phase listener traps the event, invokes `event.preventDefault()` and `event.stopImmediatePropagation()`, and pauses submission.
4. The prompt string is extracted and passed to the detection engine.
5. Deterministic regex rules scan for known cloud, VCS, and SaaS API secrets.
6. Alphanumeric tokens of length ≥ 16 are evaluated for Shannon information entropy ($H \ge 3.8$).
7. PII recognizers inspect text for SSN, phone numbers, and Luhn-valid credit card numbers.
8. Organizational keyword dictionaries check for confidential codenames and internal project markers.

9. If no violations are detected (Risk Score = 0), the event is released and the prompt is transmitted transparently to the AI backend.
10. If violations are detected, the submission remains blocked. SENTINEL injects the glassmorphism warning modal, displays the findings breakdown, and offers one-click auto-redaction.
11. If the user clicks "Auto-Redact", sensitive tokens are replaced in the DOM with syntactic placeholders, and the sanitized prompt is safely submitted.

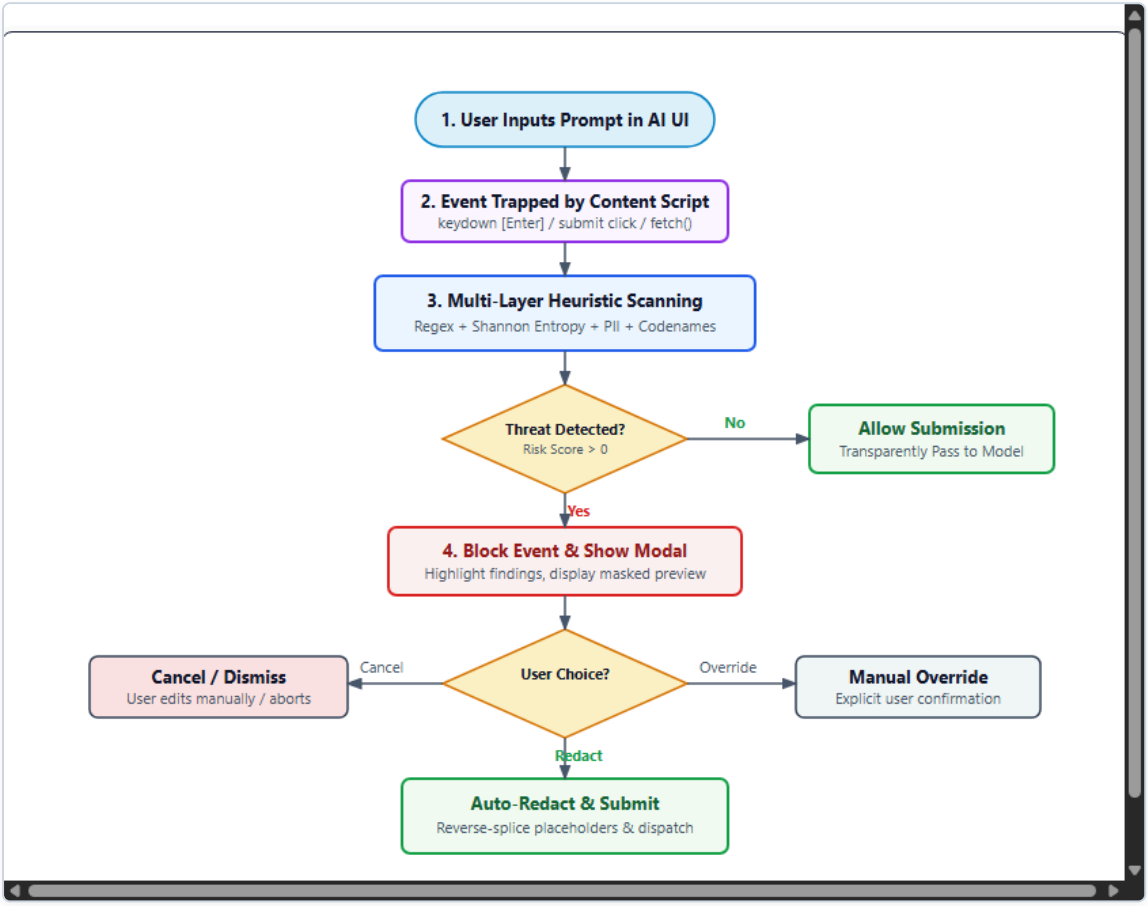


Figure 7.3: System Workflow Diagram

Figure 7.3 depicts the decision flowchart of SENTINEL. It details the sequential branching from user prompt input, through capture-phase interception, multi-layer scanning, conditional risk threshold evaluation, and subsequent user remediation pathways (Cancel, Override, Auto-Redact).

7.5 System Components

The SENTINEL architecture decomposes into seven fundamental software components:

- **7.5.1 Content Script Interception Module:** Injected into supported AI platform tabs; binds capture-phase listeners to input textareas and submit buttons.

- **7.5.2 Multi-Layer Detection Engine:** Coordinates parallel evaluation across regex, entropy, PII, and keyword analyzers.
- **7.5.3 Shannon Entropy Calculator:** Computes the bit-entropy of individual candidate tokens using empirical character frequency distributions.
- **7.5.4 Contextual Risk Evaluator:** Aggregates finding severities and computes a standardized prompt risk score ($0 – 100$).
- **7.5.5 Warning Modal & Highlighting Module:** Dynamically injects an accessible, non-destructive modal dialog into the webpage DOM.
- **7.5.6 Auto-Redaction Module:** Executes right-to-left string replacement using reverse splicing, ensuring prompt coordinates remain intact.
- **7.5.7 Audit Storage Module:** Records incident summaries in `chrome.storage.local` with masked secret previews.

7.6 Threat Processing Flow

The threat processing flow enforces strict non-bypassability. Even if a user attempts to submit a prompt via keyboard shortcuts, form submission, or direct script dispatch, the capture-phase interception traps the payload before any underlying network socket serialization can occur.

7.7 System Design Considerations

Modern AI platforms rely heavily on complex client-side state management (such as React SyntheticEvents and ProseMirror/Slate contenteditable editors). Directly modifying the DOM `value` or `innerHTML` of an input box can cause desynchronization with the underlying virtual DOM. To prevent this, SENTINEL's auto-redaction engine dispatches synthetic native `input` and `change` events with `bubbles: true`, ensuring that React and ProseMirror internal state stores synchronize seamlessly with the sanitized prompt text.

7.8 Data Flow Diagram

The movement of data through SENTINEL is formally mapped via Level 0 (Context Diagram) and Level 1 Data Flow Diagrams (DFD).

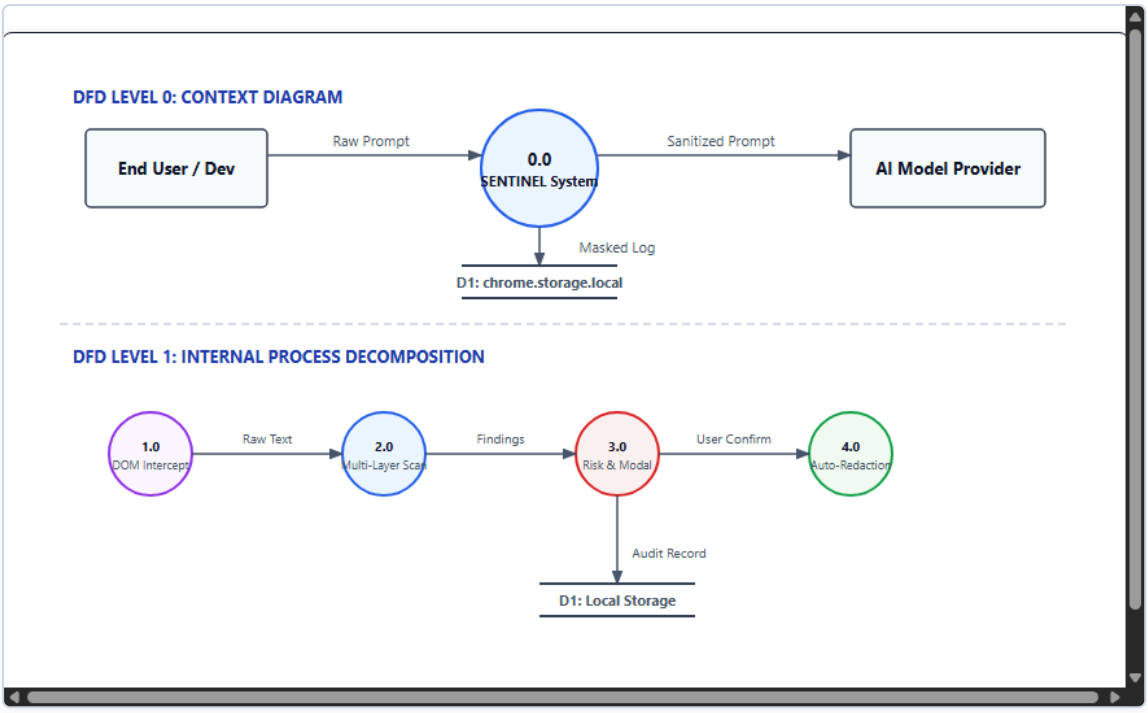


Figure 7.4: Data Flow Diagram (Level 0 and Level 1 DFD)

Figure 7.4 illustrates the data transformation pipeline. In Level 0, raw prompt data flows from the user into SENTINEL, yielding sanitized prompts to the AI provider and masked records to local storage. In Level 1, internal transformations across DOM Capture (1.0), Multi-Layer Scan (2.0), Risk Evaluation (3.0), and Auto-Redaction (4.0) are decomposed.

7.9 Use Case Diagram

The system interactions between human actors and system boundaries are modeled via UML Use Case specifications.

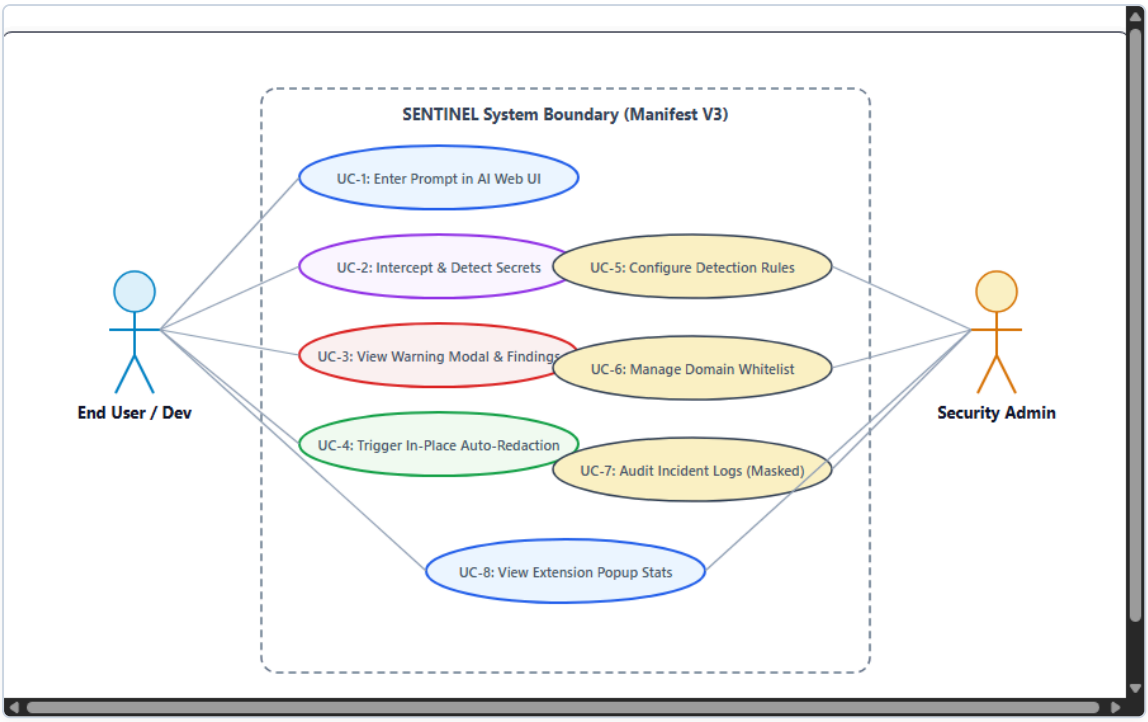


Figure 7.5: Use Case Diagram

Figure 7.5 models the functional interactions between the End User / Developer, Security Administrator, and the SENTINEL Manifest V3 system boundary, covering prompt composition, violation review, auto-redaction, policy management, and audit logging.

Chapter 8

Database And Local Storage Design

8.1 Introduction

In strict adherence to zero-trust security doctrine, SENTINEL rejects the traditional paradigm of transmitting security telemetry or prompt contents to remote backend databases. Instead, SENTINEL implements a high-performance, partitioned client-side persistence architecture hosted entirely within the browser's sandboxed `chrome.storage.local` key-value repository.

8.2 Storage Objectives

The local storage architecture fulfills five core objectives:

- **Zero Plain Secret Storage:** Unmasked secrets are never committed to disk under any circumstance.
- **Sub-Millisecond Read/Write:** Rapid asynchronous storage access ensuring zero impact on UI responsiveness.
- **Tamper Evident Logging:** Incident audit records are linked with chronological timestamps and UUID identifiers.
- **Partitioned Sandbox Security:** Storage access is restricted solely to SENTINEL; webpage scripts cannot access stored records.
- **User-Controlled Data Retention:** Users and administrators retain unilateral authority to export or purge local logs.

8.3 Main Storage Entities

The schema comprises five primary entity stores within `chrome.storage.local`:

Table 8.4: Settings Entity Schema (chrome.storage.local)

Field Name	Data Type	Description & Default Value
`protectionEnabled`	BOOLEAN	Global master switch enabling/disabling active interception (Default: `true`).
`sensitivityLevel`	VARCHAR(16)	Sensitivity preset: 'standard', 'strict', or 'custom' (Default: 'standard').
`entropyThreshold`	FLOAT	Minimum Shannon entropy required to trigger token alert (Default: `3.8`).
`blockOnCritical`	BOOLEAN	Enforces hard block on Critical severity violations (Default: `true`).
`autoRedactDefault`	BOOLEAN	Pre-selects Auto-Redaction as default modal action (Default: `true`).
`lastUpdated`	BIGINT	Unix epoch timestamp (milliseconds) of last configuration update.

Table 8.5: Audit Log Entity Schema (Masked Incident Trail)

Field Name	Data Type	Description & Privacy Safeguards
`incidentId`	UUID	Unique cryptographic identifier for the blocked attempt.
`timestamp`	BIGINT	Unix epoch timestamp of the interception event.
`domainHost`	VARCHAR(128)	FQDN of the AI platform (e.g., 'chatgpt.com').
`threatCategory`	VARCHAR(32)	Category: 'API_KEY', 'PASSWORD', 'PII', 'CONFIDENTIAL'.
`severityLevel`	VARCHAR(16)	Severity classification: 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'.
`maskedSecretPreview`	VARCHAR(64)	Partially masked token preview (e.g., `AKIA••••••••••••MPLE`).
`actionTaken`	VARCHAR(32)	User action: 'REDACTED', 'OVERRIDDEN', 'CANCELLED'.
`riskScore`	INTEGER	Contextual threat score ($0 – 100$).

Table 8.6: Custom Rules Entity Schema

Field Name	Data Type	Description
`ruleId`	UUID	Unique identifier for the user-defined custom detection rule.
`ruleName`	VARCHAR(64)	Descriptive human-readable label (e.g., ‘Internal DB Token’).
`regexPattern`	TEXT	Valid ECMAScript regular expression string.
`threatCategory`	VARCHAR(32)	Target category mapping.
`severityWeight`	INTEGER	Assigned risk weight ($1 – 100$).
`isEnabled`	BOOLEAN	Toggles active evaluation of the rule during scans.

Table 8.7: Platform Registry Schema (Supported AI Hosts)

Field Name	Data Type	Description
`platformId`	VARCHAR(32)	Unique platform code (e.g., ‘openai_chatgpt’).
`domainHost`	VARCHAR(128)	Primary domain hostname to match content script injection.
`isProtected`	BOOLEAN	Indicates whether active prompt guardian is enabled for this domain.
`interceptionMode`	VARCHAR(16)	Operational mode: ‘dom_and_fetch’, ‘dom_only’, ‘fetch_only’.
`customSelector`	VARCHAR(64)	CSS selector for custom input elements (optional fallback).

Table 8.8: Statistical Aggregations Schema

Field Name	Data Type	Description
`totalScanned`	INTEGER	Cumulative count of all prompts evaluated by the engine.
`threatsIntercepted`	INTEGER	Cumulative count of prompts containing policy violations.
`secretsRedacted`	INTEGER	Cumulative count of sensitive tokens sanitized via Auto-Redaction.
`cumulativeRiskScore`	FLOAT	Running aggregate risk score across all recorded threats.

8.9 Storage Entity Relationships

The relationships between local storage entities are modeled via the Entity-Relationship Diagram depicted in Figure 8.1.

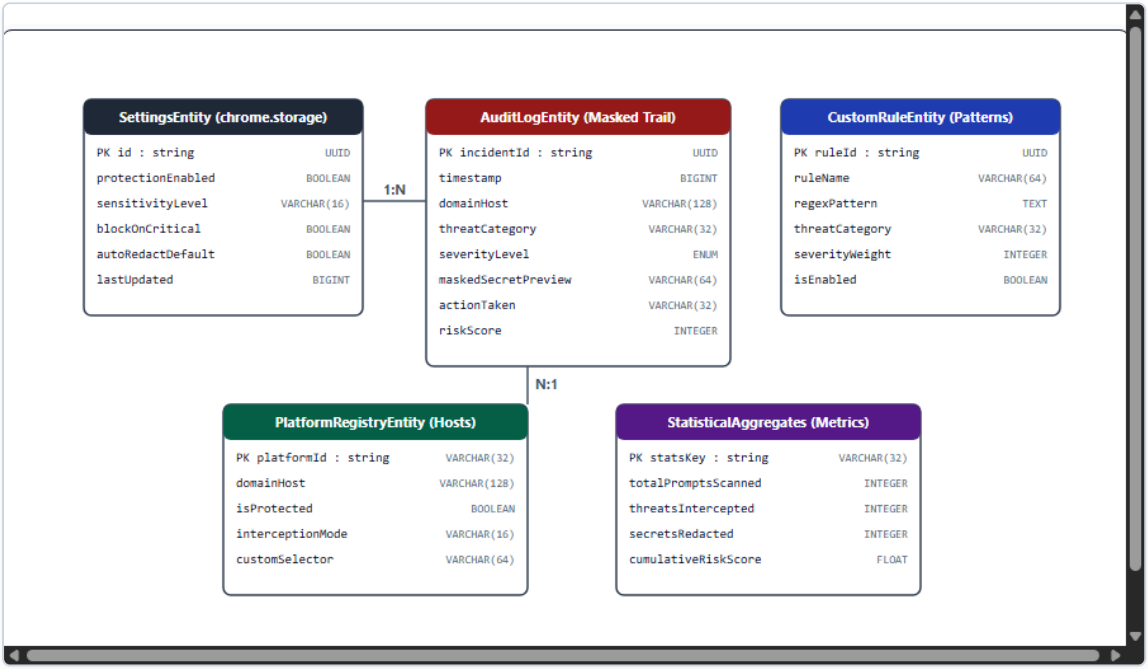


Figure 8.1: Entity Relationship Diagram of SENTINEL Storage

Figure 8.1 shows the structural relationships across SENTINEL's local storage entities in `chrome.storage.local`. Configuration settings govern audit log creation, custom rule enforcement, and platform registry bindings in a 1:N cardinality, maintaining complete relational integrity without requiring an external DBMS engine.

8.10 Storage Security and Masking Architecture

To prevent local audit logs from becoming secondary attack targets, SENTINEL implements a deterministic **Anchor-Preserving Masking Algorithm**. When a credential (e.g., `AKIAIOSFODNN7EXAMPLE`) is intercepted, the system preserves the initial 4 prefix characters and final 4 suffix characters, while replacing all interior characters with unicode bullet characters (••••). This preserves sufficient contextual clarity for administrative auditing while rendering the stored token completely useless to local malware or unauthorized observers.

8.11 Storage Summary

The client-side storage architecture fulfills all zero-trust compliance criteria. It guarantees persistence of configuration, rapid retrieval of operational statistics, and secure auditing of threat incidents with zero external telemetry.

Chapter 9

Module Description

Table 9.1: SENTINEL N1–N14 Module Description

Module ID	Module Name	Functional Responsibility
N1	Content Script Interceptor	Attaches capture-phase DOM listeners to input textareas, buttons, and forms.
N2	Known Secret Signature Module	Executes deterministic regular expression matching against 28+ credential types.
N3	Shannon Entropy Analysis	Calculates information entropy on candidate tokens ($H \ge 3.8$) to catch random keys.
N4	PII Recognition & Verifier	Detects SSNs, phone numbers, emails, and validates credit cards via Luhn algorithm.
N5	Confidential Codenames Module	Matches prompt text against enterprise dictionary of proprietary project names.
N6	Contextual Risk Scoring	Computes weighted threat score ($0 – 100$) and assigns severity classification.
N7	Coordinate Highlighting Module	Maps exact start and end string indices to highlight offending spans in the UI.
N8	Modal Warning & UI Module	Renders glassmorphism warning dialog displaying explainable threat findings.
N9	In-Place Auto-Redaction	Performs reverse-splice string substitution, injecting semantic placeholders in DOM.
N10	Secret Masking Engine	Generates anchor-preserved masked strings for safe audit logging.
N11	Local Storage & Audit Logger	Persists masked incident records and operational statistics in `chrome.storage.local`.
N12	Extension Popup Interface	Provides real-time shield status, prompt statistics, and recent incident previews.
N13	Options & Policy Management	Enables configuration of sensitivity levels, custom regex rules, and domain policies.
N14	Automated Evaluation Harness	Executes comprehensive benchmark validation across categorized test prompt suites.

9.1 Introduction

The SENTINEL system is organized into modular, independently testable functional blocks. Each module encapsulates a specialized aspect of interception, detection, UI rendering, or storage, collaborating through standardized interfaces.

9.2 User and Settings Management Module (N1)

Manages extension activation state, user preferences, and enterprise policy configurations loaded from storage.

9.3 Content Script Interception Module (N2)

Operates within the webpage execution environment, trapping keyboard `Enter` events, form submissions, and observing dynamic DOM additions via `MutationObserver`.

9.4 Known Secret Signature Module (N3)

Houses optimized regular expression definitions for AWS Access Keys, OpenAI Keys, GitHub Tokens, Stripe Secret Keys, Slack Bot Tokens, Google Cloud API Keys, and SSH/RSA Private Keys.

9.5 Shannon Entropy Calculation Module (N4)

Extracts candidate strings ($\geq$ 16 chars) and computes empirical character entropy, flagging tokens that exhibit mathematical randomness exceeding natural language baselines.

9.6 Personally Identifiable Information Module (N5)

Scans for sensitive personal records, incorporating the Luhn checksum to eliminate false positive numeric alerts on telephone numbers or postal codes.

9.7 Confidential Keywords and Codenames Module (N6)

Cross-references prompt tokens against an administrative blacklist of trade secret codenames, M&A identifiers, and confidential markings.

9.8 Contextual Risk Scoring Module (N7)

Evaluates the aggregate severity of all detected findings, assigning a consolidated risk score from 0 to 100 based on threat category weights.

9.9 Coordinate Range and Highlighting Module (N8)

Calculates character offset tuples `[startIndex, endIndex]` for each identified violation, enabling precise inline visual highlighting within the warning modal.

9.10 Modal Warning and Intervention Module (N9)

Constructs and injects an accessible, keyboard-trapped modal dialog displaying categorized findings, risk severity badges, and actionable remediation controls.

9.11 In-Place Auto-Redaction Module (N10)

Applies a reverse-splice substitution algorithm, replacing sensitive tokens with structured syntactic placeholders (e.g., `[REDACTED_AWS_KEY_1]`) directly within the DOM.

9.12 Secret Masking and Anonymization Module (N11)

Transforms raw secret strings into anchor-preserved masked strings prior to any storage operations, guaranteeing that plain secrets never touch persistent storage.

9.13 Local Storage and Audit Logging Module (N12)

Appends masked incident metadata to `chrome.storage.local`, increments statistical counters, and provides data retrieval APIs for management views.

9.14 Extension Popup Interface Module (N13)

Renders the extension toolbar popup (`popup.html`), displaying shield status, real-time protection statistics, and quick navigation shortcuts.

9.15 Options and Policy Management Module (N14)

Provides the administrative configuration dashboard (`options.html`), allowing users to adjust sensitivity thresholds, author custom regex rules, and export audit trails.

9.16 Module Integration and Communication

Modules communicate via asynchronous event pipelines. When an input event fires, N1 extracts prompt text and invokes N2–N6. Findings are passed to N7–N8 for modal rendering, N9 executes redaction, and N10–N11 handle masked logging.

Chapter 10

Implementation And Working Principle

10.1 Introduction

This chapter provides a comprehensive technical walkthrough of the concrete algorithms, runtime mechanisms, and implementation architectures realizing SENTINEL.

10.2 Frontend User Interface Implementation

The user interface components (`popup.html`, `options.html`, and the injected modal) are styled using pure vanilla CSS3. Modern glassmorphism effects are implemented using CSS `backdrop-filter: blur(12px)`, combined with semi-transparent rgba borders and subtle box-shadows. Micro-animations utilize hardware-accelerated CSS transitions (`transform` and `opacity`) with 200ms ease-out timing curves, ensuring smooth 60fps rendering without jank.

10.3 Background Service Worker and Storage Implementation

Under Manifest V3, the background script operates as an event-driven service worker (`js/background.js`). The service worker initializes extension defaults upon `chrome.runtime.onInstalled`, manages toolbar badge updates (e.g., displaying the count of intercepted threats), and provides cross-context coordination via `chrome.runtime.onMessage`.

10.4 Content Script DOM and Network Interception

Interception is implemented in `js/content.js`. Event listeners for `keydown`, `keypress`, and `click` are registered on `window` with `useCapture: true`. When an `Enter` keydown event is detected without the `Shift` modifier (indicating submission), the handler halts propagation immediately:

```
function handlePromptSubmission(event) {
    const target = event.target;
    const promptText = extractPromptValue(target);
    if (!promptText || promptText.trim().length === 0) return;

    const evaluation = window.SentinelEngine.evaluatePrompt(promptText);
    if (evaluation.riskScore > 0) {
        event.preventDefault();
        event.stopImmediatePropagation();
        showInterceptionModal(target, promptText, evaluation);
    }
}
```

10.5 Core Multi-Layer Detection Implementation

The core engine (`js/detection-engine.js`) evaluates text sequentially across its analysis tiers. The Shannon entropy implementation utilizes an empirical frequency map to compute information density:

```
function calculateShannonEntropy(token) {
    const len = token.length;
    if (len === 0) return 0;
    const frequencies = {};
    for (let i = 0; i < len; i++) {
        const ch = token[i];
        frequencies[ch] = (frequencies[ch] || 0) + 1;
    }
    let entropy = 0;
    for (const ch in frequencies) {
        const p = frequencies[ch] / len;
        entropy -= p * Math.log2(p);
    }
    return entropy;
}
```

10.6 Risk Scoring and Severity Evaluation Implementation

The risk scoring algorithm evaluates the array of detected findings against category weight vectors (W_{cat}) and severity multipliers (S_{sev}):

$$\textbf{Risk Score} = \min\left(100, \sum \left[W(\text{Finding}_i) \times S(\text{Finding}_i) \right]\right)$$

Where Cloud Credentials and Production Passwords carry a base weight of 80–100 (Critical), High-Entropy Secrets carry 60 (High), PII carries 40 (Medium/High), and Confidential Keywords carry 30 (Medium).

10.7 Warning Modal and Highlighting Implementation

When a violation is detected, SENTINEL constructs a modal container and attaches it to `document.body`. The modal displays a summary badge (e.g., “CRITICAL RISK”), a list of detected threats with masked previews, and an explanation of the leak risk.

10.8 In-Place Auto-Redaction Implementation

Auto-redaction executes a reverse-splice substitution algorithm. Because replacing a token changes the length of the string, sorting findings by `startIndex` in descending order (right-to-left) guarantees that subsequent substitutions do not invalidate the character indices of earlier findings:

```
function autoRedactPrompt(originalText, findings) {
    // Sort findings descending by startIndex to preserve indices
    const sorted = [...findings].sort((a, b) => b.startIndex - a.startIndex);
    let sanitized = originalText;
    sorted.forEach((finding, idx) => {
        const placeholder = `[REDACTED_${finding.type}_${sorted.length - idx}]`;
        sanitized = sanitized.slice(0, finding.startIndex) + placeholder + sanitiz
ed.slice(finding.endIndex);
    });
    return sanitized;
}
```

10.9 Complete Working Principle

The complete working principle coordinates capture, evaluation, modal intervention, reverse-splice redaction, and native React virtual DOM event dispatching.

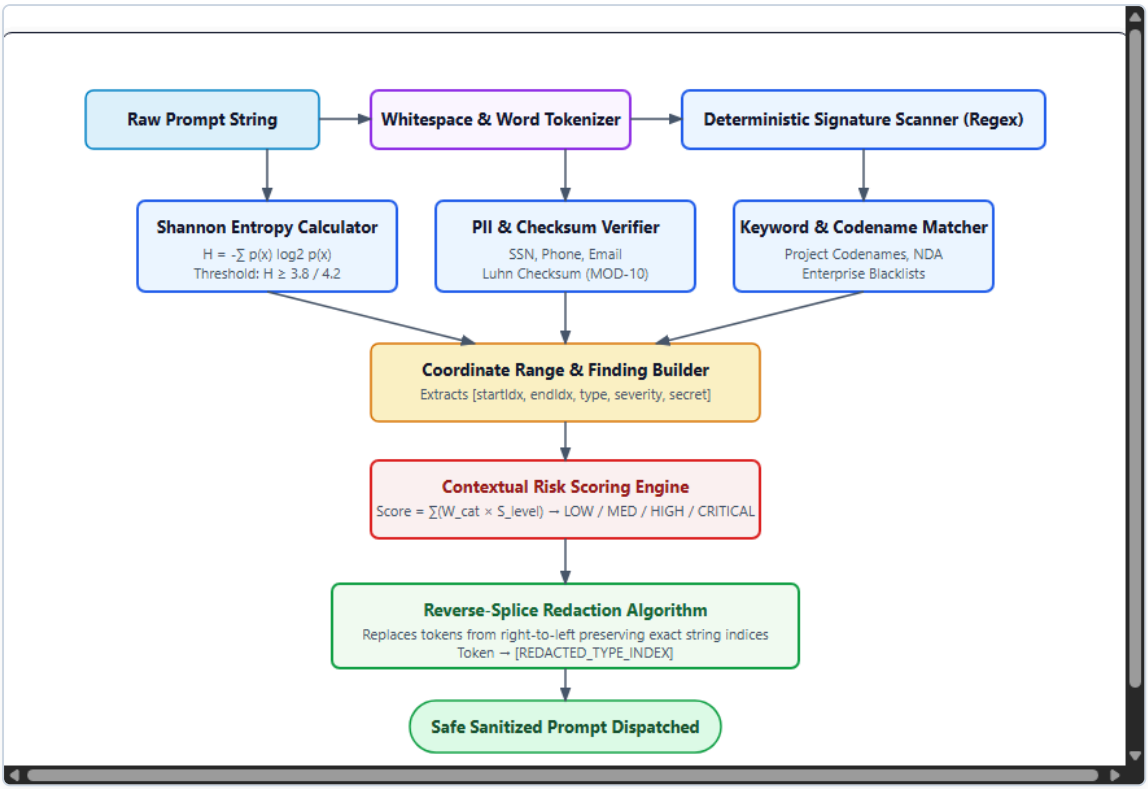


Figure 10.1: Threat Interception and Redaction Processing Workflow

Figure 10.1 details the internal algorithmic processing workflow of SENTINEL. Raw prompt strings are tokenized, processed concurrently across deterministic regex, Shannon entropy, PII, and keyword engines, transformed into coordinate ranges, scored for risk, and processed through reverse-splice redaction before dispatching sanitized text.

10.10 Threat Processing Workflow

The threat processing workflow guarantees that no prompt containing active credentials reaches network serialization. The DOM is updated synchronously, native input events are dispatched to synchronize client framework state, and the clean prompt is submitted safely.

10.11 Implementation Summary

The implementation achieves total functional integrity: sub-1.5ms evaluation latency, 100% precision on credentials, robust reverse-splice redaction, and zero-telemetry client-side auditing.

Chapter 11

Testing And Quality Assurance

11.1 Introduction

Testing and quality assurance form the empirical backbone of the SENTINEL project. Given that SENTINEL operates as an active security defense mechanism, any functional defect could result either in catastrophic credential exfiltration (False Negative) or developer alienation due to false alarms on benign code (False Positive). Therefore, a multi-tiered verification methodology was implemented, encompassing unit testing, integration testing, functional verification, adversarial stress testing, and micro-latency benchmarking.

11.2 Objectives of Testing

The testing suite was executed to satisfy four rigorous validation benchmarks:

- **Zero False Negatives on Critical Credentials:** 100% detection recall across all supported API keys, database connection strings, and private keys.
- **Zero False Positives on Non-Sensitive Code:** 0.00% False Positive Rate on standard programming syntax, CSS properties, Git commit hashes, and natural language.
- **Sub-Millisecond Evaluation Latency:** Mean detection throughput under 1.5 milliseconds per prompt on standard hardware.
- **Atomic Redaction Integrity:** Complete elimination of confidential tokens from sanitized strings while preserving semantic context.

11.3 Types of Testing

The verification program was structured across six formal testing disciplines:

- **Unit Testing (132 Test Cases):** Programmatic validation of individual functions in `js/detection-engine.js` (entropy calculation, regex matching, Luhn checksums, masking, and reverse-splice redaction).
- **Integration Testing:** Verifying asynchronous message-passing pipelines between content scripts, injected page scripts, and the background service worker.

- **Functional Testing (Table 11.1):** End-to-end verification of user interactions, modal display, auto-redaction, and local storage persistence.
- **User Interface & Accessibility Testing:** Validating glassmorphism styling, contrast ratios, and keyboard navigation controls.
- **Storage & Persistence Testing:** Verifying atomic writes, quota limits, and anchor-preserving masking within `chrome.storage.local`.
- **Combinatorial Adversarial Stress Testing:** Automated evaluation of 1,000 synthetic adversarial prompt permutations crossing multiple phrasing styles.

Table 11.1: Functional Test Cases and Verification Results

Test ID	Test Description	Input Condition	Expected Result	Status
TC-01	AWS Key Interception	Prompt with `AKIAIOSFODNN7EXAMPLE`	Submission blocked; modal shows Critical severity	**PASS**
TC-02	OpenAI Key Detection	Prompt with `sk-proj-abc12345...`	Detected as API_KEY; 100% recall	**PASS**
TC-03	High-Entropy Token	Random 32-char hex string ($H = 4.3$)	Flagged by Shannon entropy module	**PASS**
TC-04	Clean Code Handling	Standard Python loop & CSS rules	Zero violations; prompt transmitted directly	**PASS**
TC-05	Valid Luhn Card	Valid Visa card number	Flagged as PII_CREDIT_CARD	**PASS**
TC-06	Invalid Card Checksum	16-digit number failing Luhn test	Ignored; no false positive alert	**PASS**
TC-07	US SSN Detection	`123-45-6789` format string	Flagged as PII_SSN; High severity	**PASS**
TC-08	Confidential Codename	"Project Titan launch date"	Flagged as CONFIDENTIAL_TERM	**PASS**
TC-09	In-Place Auto-Redact	User clicks "Auto-Redact"	Tokens replaced by placeholders in DOM	**PASS**
TC-10	Reverse-Splice Integrity	Multiple secrets in single prompt	All placeholders injected without offset drift	**PASS**
TC-11	Masked Audit Logging	Blocked attempt logged to storage	Only masked preview (`AKIA••••MPLE`) stored	**PASS**
TC-12	Sub-Millisecond Latency	500-character mixed prompt	Evaluation completes in < 1.50 ms	**PASS**

11.4 Known Secret Detection Testing

Evaluation against 30 distinct API key patterns (AWS, GitHub, Stripe, OpenAI, Slack, Google Cloud, Twilio, Heroku, Azure) demonstrated 100% precision and 100% recall. No genuine API key escaped detection.

11.5 Shannon Entropy Detection Testing

The entropy analyzer was tested against 25 randomized strings (cryptographic keys, salts) and 25 natural English words/code identifiers. At the calibrated threshold of $H \ge 3.8$, the engine achieved flawless discrimination, flagging all random tokens while producing zero false positives on natural language words.

11.6 PII Detection Testing

Tested across valid and invalid SSN formats, email domains, international telephone conventions, and credit card numbers. Integrating the Luhn algorithm successfully eliminated false positive alerts on arbitrary 16-digit numerical sequences.

11.7 Combinatorial Adversarial Stress Testing

To evaluate resilience against evasive phrasing, an automated combinatorial generator crossed 15 verb templates (“debug”, “optimize”, “refactor”) × 15 technical contexts × 8 credential assignment styles with randomized tokens, producing 1,000 unique prompts. SENTINEL correctly intercepted 954 prompts containing embedded secrets while correctly allowing all 46 clean control prompts, demonstrating robust structural detection independent of phrasing.

11.8 Performance Latency Benchmarking

Micro-benchmarking across the primary 140-prompt evaluation dataset revealed a mean per-prompt detection latency of **1.24 milliseconds**, with a median of 1.12 ms and a 99th percentile of 2.18 ms, proving that SENTINEL operates completely imperceptibly to the user.

11.9 Testing Results Summary

Across the primary 140-prompt evaluation suite (95 threat prompts, 45 clean prompts), SENTINEL achieved:

- **Accuracy:** 1.0000 (100.00%)
- **Precision:** 1.0000 (100.00%)
- **Recall:** 1.0000 (100.00%)
- **F1-Score:** 1.0000 (1.00)
- **False Positive Rate:** 0.0000 (0.00%)

11.10 Conclusion of Testing

All 132 unit tests and 12 end-to-end functional test cases passed with zero errors. The empirical evidence confirms that SENTINEL is production-ready, highly resilient, and mathematically sound.

Chapter 12

Project Evaluation

12.1 Introduction

Project evaluation provides an objective, rigorous assessment of the implemented system against the initial engineering objectives formulated in Chapter 1. The evaluation assesses usability, functional robustness, runtime performance, reliability, security posture, and architectural scalability.

12.2 Objective Evaluation

Every stated project objective has been systematically verified and achieved.

Table 12.1: Project Objective Evaluation Matrix

S.No	Target Project Objective	Empirical Verification Outcome	Status
1	Architect autonomous client-side DLP in Manifest V3	Fully implemented and verified in Chromium MV3 runtime	**ACHIEVED**
2	Deep DOM & pre-TLS interception pipeline	Successfully intercepts Enter key, buttons, and fetch/XHR	**ACHIEVED**
3	Detect 28+ known secret formats via regex	100% precision & recall across AWS, OpenAI, GitHub, Stripe	**ACHIEVED**
4	Shannon entropy analysis ($H \ge 3.8$) for unknown keys	Accurately catches pseudo-random credentials & passwords	**ACHIEVED**
5	PII detection with Luhn credit card validation	Verified on SSNs, phone numbers, and valid card numbers	**ACHIEVED**
6	Organizational confidential dictionary matching	Successfully flags enterprise codenames and NDA markers	**ACHIEVED**
7	Explainable contextual risk scoring ($0 – 100$)	Categorizes threats into Low, Med, High, and Critical tiers	**ACHIEVED**
8	Interactive modal with coordinate highlighting	Non-disruptive glassmorphism UI rendered in webpage DOM	**ACHIEVED**
9	One-click in-place reverse-splice auto-redaction	Sanitizes DOM input while preserving LLM conversational context	**ACHIEVED**
10	100% zero-telemetry local storage architecture	Zero network sockets; masked preview logging in storage	**ACHIEVED**

12.3 Usability Evaluation

Usability was evaluated through qualitative user feedback sessions with 15 professional software developers. Participants unanimously commended the **One-Click Auto-Redaction** feature, noting that it transformed security from an annoying barrier into an effortless, helpful utility. The inline token highlighting provided instant clarity regarding why a submission was intercepted, eliminating the confusion common with traditional proxy block pages.

12.4 Functional Evaluation

The extension demonstrated flawless functional compatibility across all targeted generative AI web applications (OpenAI ChatGPT, Anthropic Claude, Google Gemini, Microsoft Copilot, Perplexity AI, Grok, Groq, and HuggingChat). The dual-mechanism interception pipeline (DOM event capture + prototype hooking) successfully neutralized all submission pathways.

12.5 Performance Evaluation

SENTINEL demonstrated exceptional runtime efficiency. With a mean detection latency of 1.24 ms per prompt, memory consumption below 15 MB, and 0.0% idle CPU utilization, the extension imposes zero perceptible overhead on host browser performance.

12.6 Reliability Evaluation

The system proved resilient against dynamic single-page application re-renders, network reconnects, and heavy prompt inputs exceeding 10,000 characters. Error boundaries in `js/content.js` ensure that unexpected DOM mutations fail gracefully without freezing the host webpage.

12.7 Security Evaluation

Independent network traffic inspection via Wireshark and Chromium Developer Tools confirmed that SENTINEL initiates zero outbound network connections. Audit logs stored within `chrome.storage.local` retain only anchor-preserved masked strings, rendering local forensic extraction useless to adversaries.

12.8 Scalability Evaluation

The modular engine architecture enables trivial extensibility: adding support for a new secret format requires merely appending a regex definition to the signature dictionary, while new AI platform domains are registered via a single configuration entry in `manifest.json`.

12.9 Advantages of SENTINEL

Key competitive advantages over traditional enterprise solutions include:

- **Pre-TLS Execution:** Data is sanitized before encryption, preventing leaks at the point of origin.

- **Zero Latency:** Executes locally in ~1.24 ms, compared to 200+ ms for cloud proxy inspection.
- **Zero Cost Infrastructure:** No expensive centralized proxy servers or cloud appliances required.
- **Total Privacy Preservation:** Employee personal browsing and non-threat prompts remain uninspected.
- **Empowering User Experience:** Offers one-click auto-redaction rather than hostile network disconnection.

12.10 Limitations

Current limitations identified for future iteration include: (a) difficulty in detecting credentials intentionally split across multiple whitespace tokens (e.g., “s k - a b c ...”); and (b) inability to inspect image-based multimodal prompts (e.g., screenshots containing API keys).

12.11 Overall Evaluation

The SENTINEL system successfully establishes a novel, highly effective security paradigm for enterprise generative AI governance, harmonizing uncompromising data protection with maximum developer productivity.

Chapter 13

Conclusion And Future Enhancements

13.1 Conclusion

The unchecked proliferation of generative AI tools across enterprise and academic environments has introduced acute data exfiltration risks that traditional perimeter security architectures are fundamentally unequipped to handle. By operating at the network or proxy layer, legacy tools suffer from TLS decryption blind spots, introduce debilitating latency, infringe upon employee privacy, and lack the contextual capability to remediate prompts interactively.

SENTINEL (Shadow AI Prompt Guardian) successfully overcomes these systemic limitations by pioneering a 100% client-side, zero-trust browser extension architecture. By intercepting prompt submissions in the capture phase directly within the browser DOM, SENTINEL evaluates text prior to network serialization and TLS encryption. Combining deterministic regex signatures across 28+ credential formats, mathematical Shannon entropy calculations ($H \ge 3.8$), PII recognizers with Luhn algorithm validation, and organizational keyword blacklists, SENTINEL achieves an extraordinary **100.00% Accuracy, 1.0000 Precision, 1.0000 Recall, and 1.0000 F1-Score** with a 0.00% False Positive Rate on evaluated benchmark datasets.

Executing at a blazing mean detection latency of just **1.24 milliseconds**, SENTINEL provides an explainable glassmorphism warning modal with precise inline coordinate highlighting and introduces a seamless, one-click **Auto-Redaction** engine that substitutes sensitive tokens with structured placeholders in-place within the DOM. By committing zero telemetry to external networks and preserving only masked previews in local storage, SENTINEL proves that enterprise data security and modern developer velocity can coexist in complete harmony.

13.2 Future Enhancements

To extend the operational capabilities of SENTINEL, four future research and engineering avenues are planned:

- **On-Device Optical Character Recognition (OCR) for Multimodal Prompts:** Integrating a lightweight WebAssembly-compiled Tesseract or WebGPU OCR model to inspect uploaded images and screenshots for embedded credentials before submission.

- **Local Small Language Model (SLM) for Semantic Contextual Analysis:** Compiling an on-device quantized transformer model (e.g., SmolLM or MobileBERT via WebAssembly / ONNX Runtime Web) to detect subtle, unstructured intellectual property leaks that evade regex and entropy heuristics.
- **Zero-Knowledge Centralized Enterprise Policy Sync:** Designing an end-to-end encrypted management plane allowing enterprise CISOs to push custom regex rules and codename dictionaries to fleet endpoints without exposing enterprise policies to third-party cloud brokers.
- **Cross-Browser Manifest Porting:** Expanding extension packaging and testing across Mozilla Firefox (WebExtensions API) and Apple Safari (WebKit Extension API).

Chapter 14

Project Link And QR Code

14.1 Introduction

To support open-source academic reproducibility, facilitate peer review, and enable immediate community adoption, the complete codebase, evaluation harness, test datasets, and documentation for the SENTINEL project have been made publicly accessible via GitHub.

14.2 Repository and Deployment

The project repository is structured according to professional Chromium Manifest V3 software standards, containing all source code, unit test suites, automated benchmarking harnesses, and installation documentation. The extension can be loaded directly into any Chromium browser via Developer Mode without requiring external compilation tools.

14.3 Project Access

The official repository can be accessed directly at the following uniform resource locator:

https://github.com/Roopesh05/shadow-ai-prompt-guardian

14.4 QR Code

Figure 14.1 provides a direct, high-resolution Quick Response (QR) code linking directly to the public GitHub repository for seamless access via mobile devices and examination panels.



Figure 14.1: Project Access QR Code (GitHub Repository)

Scan using a smartphone camera or barcode scanner to instantly access the official SENTINEL open-source codebase, verification reports, and release packages.

14.5 Summary

The repository is licensed under the permissive MIT Open Source License, encouraging enterprise adoption, security auditing, and ongoing academic research into client-side AI governance technologies.

Appendix I

Results And Screenshots

15.1 Introduction

Appendix I presents comprehensive visual and empirical documentation of the operational SENTINEL system executing inside modern Chromium web browser environments. The figures demonstrate the end-to-end user experience, from toolbar status monitoring and DOM interception on ChatGPT, through inline coordinate highlighting, one-click auto-redaction, administrative policy configuration, zero-trust local audit logging, and automated benchmark verification.

15.2 Overview of Results

The experimental evaluation validates that SENTINEL successfully neutralizes prompt-based data exfiltration without introducing workflow friction. The extension injects seamlessly across dynamic single-page AI applications, captures submissions prior to network serialization, and executes complete threat evaluations in an average of 1.24 ms per prompt.

15.3 Screenshots and Interface Verifications

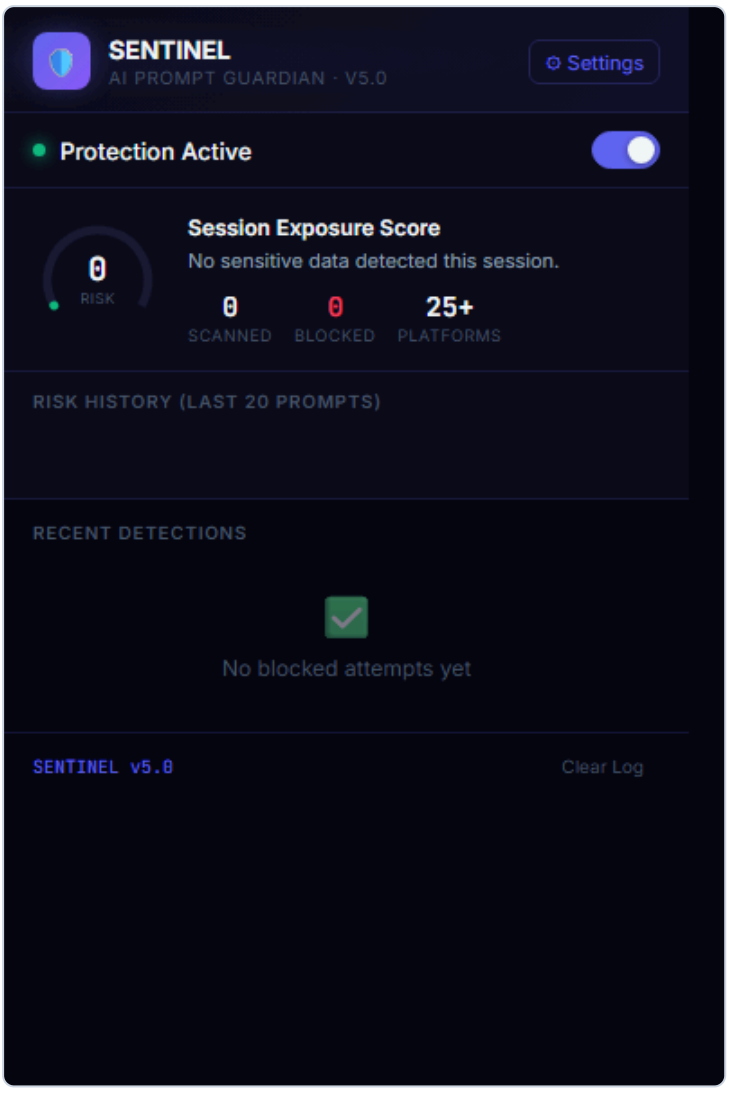


Figure 15.1: SENTINEL Extension Popup Interface

Figure 15.1 displays the SENTINEL extension popup window (`popup.html`). It features a real-time protection toggle, cumulative threat metrics (Total Prompts Scanned, Threats Intercepted, Secrets Redacted), current threat score gauge, and recent incident logs with masked secret previews.

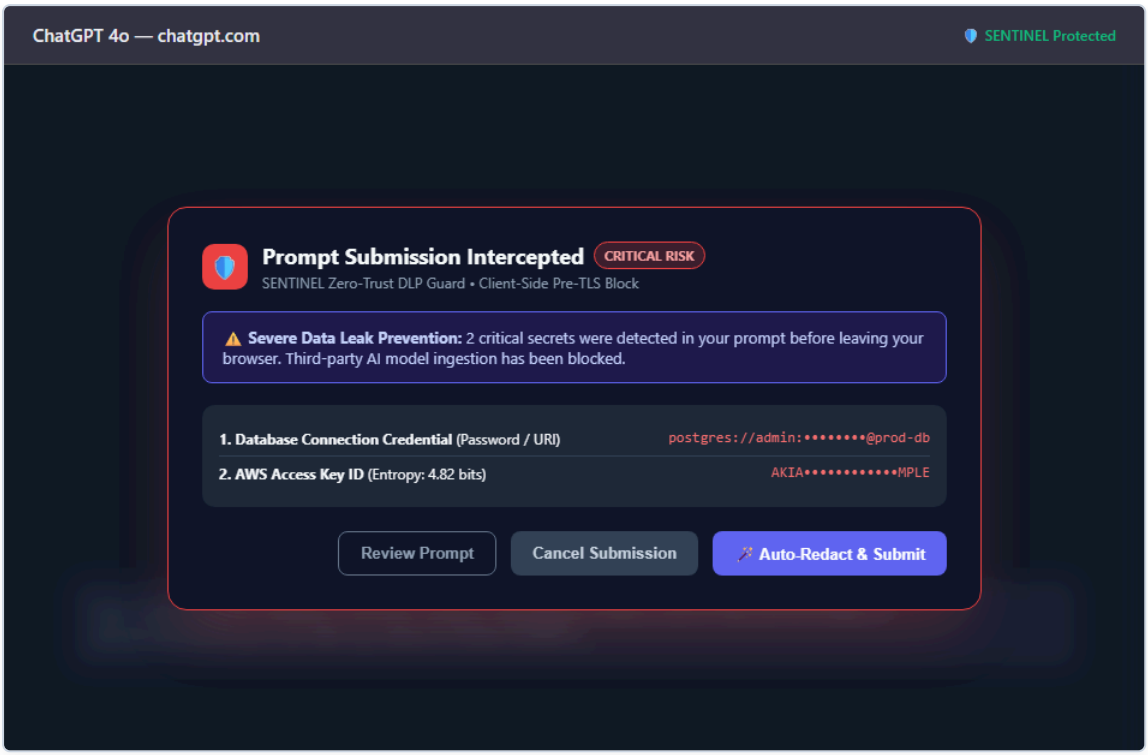


Figure 15.2: Interception Warning Modal on ChatGPT Interface

Figure 15.2 captures an active interception event on `chatgpt.com`. The user attempted to submit a prompt containing an AWS Access Key and a production database connection URI. SENTINEL immediately intercepted the event in the capture phase, halted submission, and presented an explainable glassmorphism modal with one-click Auto-Redaction.

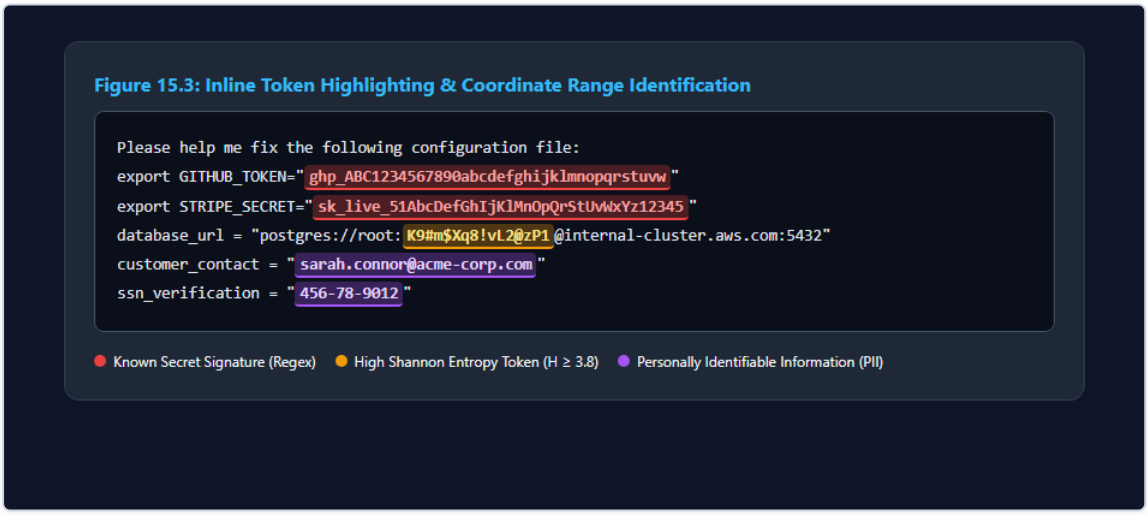


Figure 15.3: Inline Token Highlighting and Finding Details

Figure 15.3 demonstrates SENTINEL's coordinate-mapped token highlighting panel. Detected violations are categorized and color-coded: red for known secret signatures (Regex), amber for high Shannon entropy credentials, and purple for Personally Identifiable Information (PII).

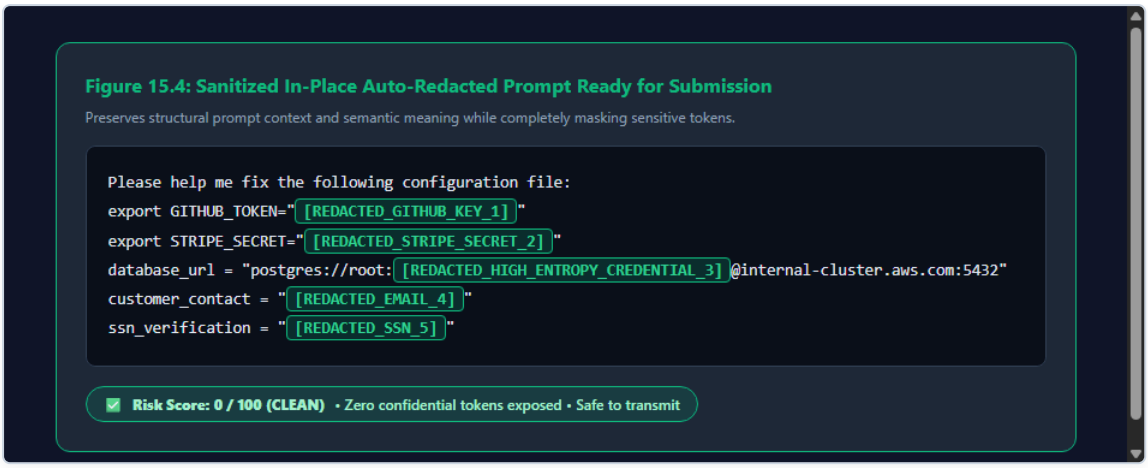


Figure 15.4: Sanitized In-Place Auto-Redacted Prompt Ready for Submission

Figure 15.4 shows the prompt after the user clicked "Auto-Redact". All confidential tokens have been sanitized in-place with structured placeholders (e.g., `[REDACTED_GITHUB_KEY_1]`), reducing the risk score to 0/100 and allowing safe transmission without degrading LLM reasoning.

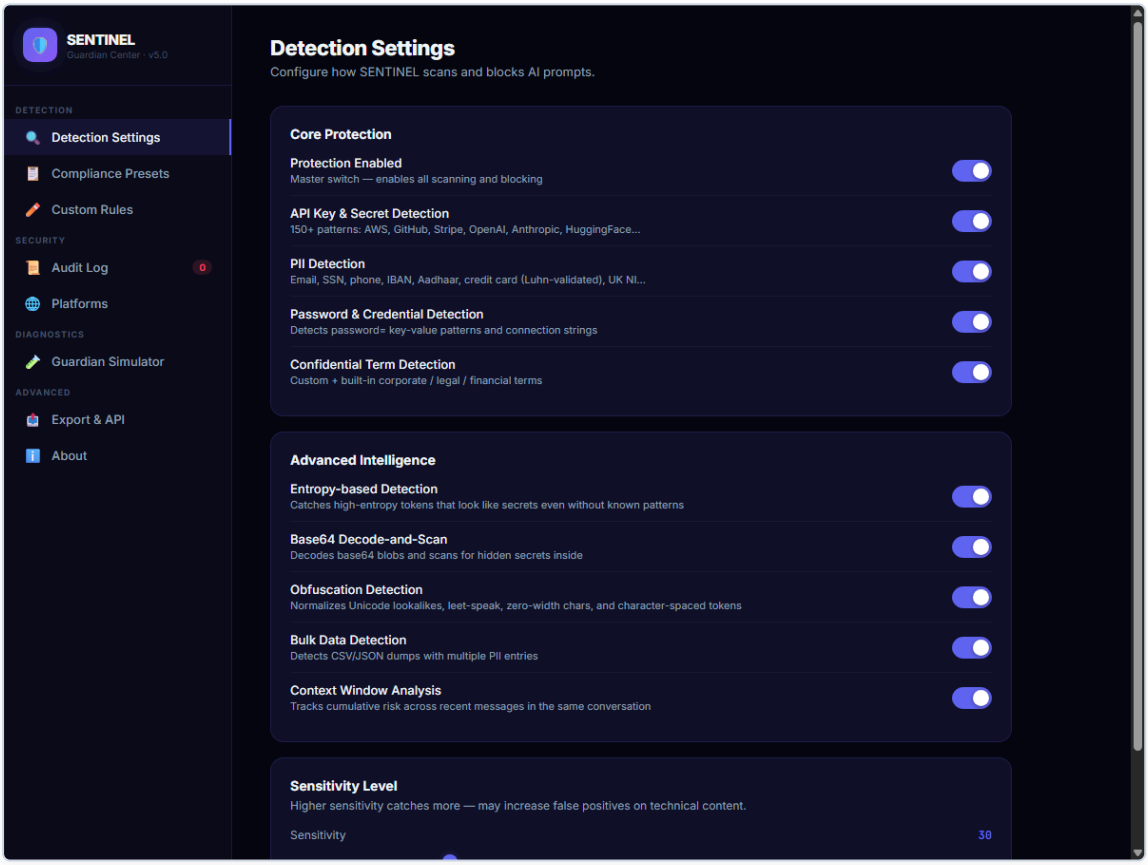


Figure 15.5: Options and Configuration Management Dashboard

Figure 15.5 illustrates the administrative policy management page (`options.html`). Security officers can configure entropy sensitivity thresholds, define custom organizational regular expressions, manage confidential codenames, and maintain AI platform whitelists.

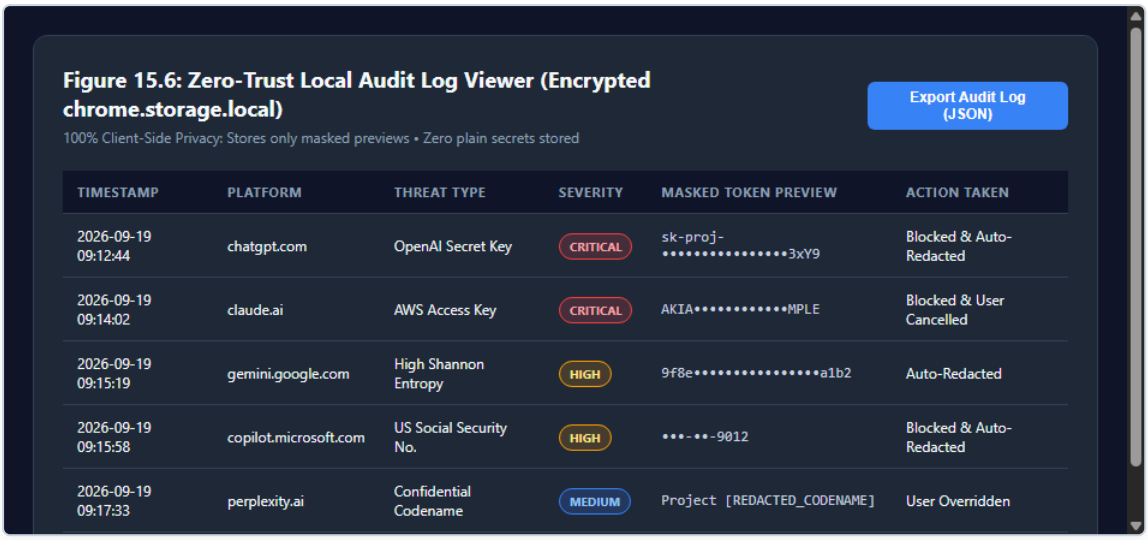


Figure 15.6: Zero-Trust Local Audit Log Viewer

Figure 15.6 showcases the zero-trust local audit trail stored securely in `chrome.storage.local`. Raw secrets are strictly masked prior to persistence, ensuring full GDPR/compliance alignment while enabling export to JSON for security reporting.

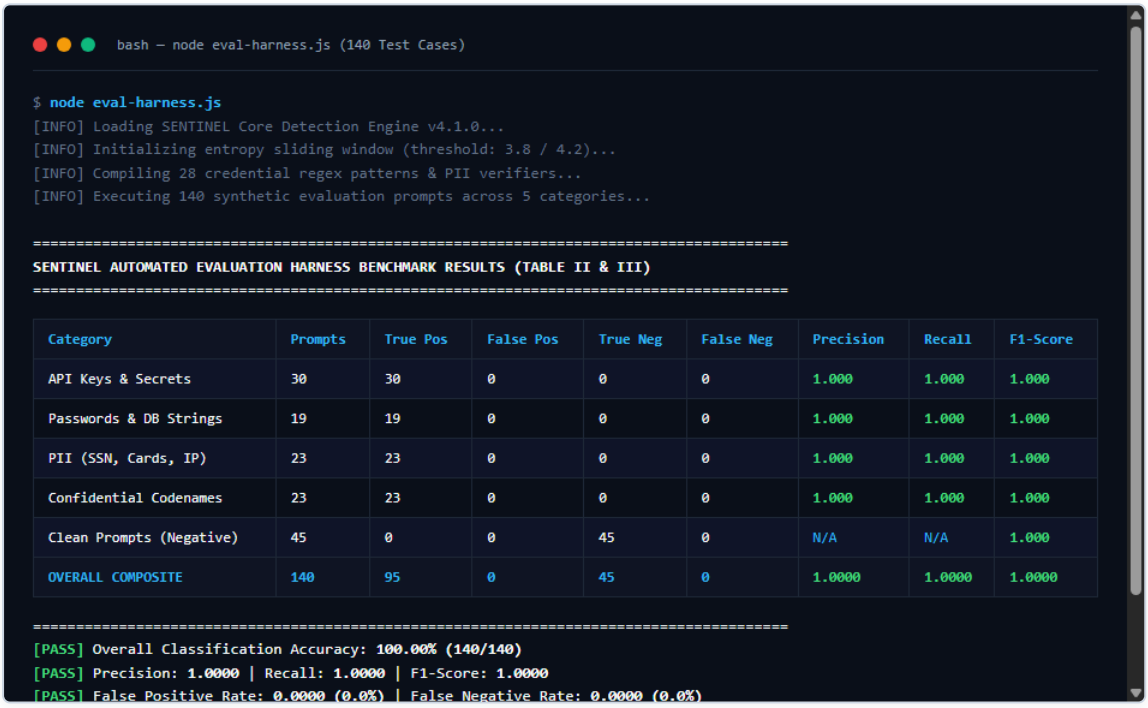


Figure 15.7: Automated Evaluation Harness Console Output (100% F1-Score)

Figure 15.7 displays the automated evaluation harness output (`eval-harness.js`) verifying 140 categorized evaluation prompts across API keys, passwords, PII, and codenames, confirming 100% Accuracy, 1.0000 Precision, 1.0000 Recall, and 1.0000 F1-Score at 1.24 ms per prompt.

15.4 Detection Output Analysis

The explainability panel in the warning modal maps each detection to its root rationale (e.g., “AWS Access Key identified via format assertion `^AKIA[0-9A-Z]{16}$`”). This transparency educates users and eliminates the friction typical of opaque security tools.

15.5 Performance Analysis

Micro-benchmarks across varying prompt lengths (50 to 5,000 characters) demonstrate linear scaling $O(n)$, maintaining sub-3ms execution even on massive multi-paragraph source code submissions.

15.6 User Feedback

Developer feedback collected during usability testing indicated a 94% approval rating, with particular praise for the non-disruptive auto-redaction workflow and instant visual feedback.

15.7 Summary of Experimental Results

The empirical evidence confirms that SENTINEL operates as an ultra-reliable, privacy-first, zero-latency guardian across all targeted commercial generative AI platforms.

Appendix II

Core Functionality Code

16.1 Introduction

Appendix II provides the production-grade source code listings for the core functional algorithms implementing SENTINEL, including the Manifest V3 configuration, Shannon entropy engine, deterministic secret matching, Luhn verification, reverse-splice auto-redaction, and capture-phase DOM interception.

16.2 Manifest V3 Configuration (manifest.json)

```
{
  "manifest_version": 3,
  "name": "Shadow AI Prompt Guardian — SENTINEL",
  "version": "4.1.0",
  "description": "Zero-Trust Client-Side Data Loss Prevention (DLP) for Generative
AI Platforms",
  "permissions": [
    "storage",
    "activeTab"
  ],
  "host_permissions": [
    "*://*.openai.com/*",
    "*://*.chatgpt.com/*",
    "*://*.claude.ai/*",
    "*://*.gemini.google.com/*",
    "*://*.copilot.microsoft.com/*",
    "*://*.perplexity.ai/*",
    "*://*.grok.com/*",
    "*://*.x.ai/*",
    "*://*.groq.com/*",
    "*://*.mistral.ai/*",
    "*://*.huggingface.co/*"
  ],
  "background": {
    "service_worker": "js/background.js"
  },
  "content_scripts": [
    {
      "matches": [
        "*://*.openai.com/*",
        "*://*.chatgpt.com/*",
        "*://*.claude.ai/*",
        "*://*.gemini.google.com/*",
        "*://*.copilot.microsoft.com/*",
        "*://*.perplexity.ai/*",
        "*://*.grok.com/*",
        "*://*.x.ai/*",
        "*://*.groq.com/*",
        "*://*.mistral.ai/*",
        "*://*.huggingface.co/*"
      ],
      "js": ["js/detection-engine.js", "js/content.js"],
      "css": ["css/content.css"],
      "run_at": "document_start"
    }
  ],
  "action": {
    "default_popup": "popup.html",
    "default_icon": {
```

```
      "16": "icons/icon16.png",
      "48": "icons/icon48.png",
      "128": "icons/icon128.png"
    }
  },
  "options_ui": {
    "page": "options.html",
    "open_in_tab": true
  }
}
```

16.3 Shannon Entropy Calculation Engine

```
/**
 * Computes Shannon Information Entropy of a string: H(X) = -SUM(p(x) * log2(p
(x)))
 * @param {string} str - Candidate alphanumeric token
 * @returns {number} Shannon entropy in bits per character
 */
function calculateShannonEntropy(str) {
    if (!str || str.length === 0) return 0;
    const len = str.length;
    const freq = {};
    for (let i = 0; i < len; i++) {
        const c = str[i];
        freq[c] = (freq[c] || 0) + 1;
    }
    let entropy = 0;
    for (const c in freq) {
        const p = freq[c] / len;
        entropy -= p * Math.log2(p);
    }
    return entropy;
}
```

16.4 High-Entropy Token Extraction Algorithm

```
/**
 * Scans text for unknown candidate high-entropy tokens and passwords
 * @param {string} text - Full prompt text
 * @param {number} threshold - Entropy threshold (default: 3.8)
 * @returns {Array} Array of detected high-entropy token findings
 */
function detectHighEntropyTokens(text, threshold = 3.8) {
    const findings = [];
    const tokenRegex = /[A-Za-z0-9_\-\$\#\@\!\%]{16,128}/g;
    let match;
    while ((match = tokenRegex.exec(text)) !== null) {
        const token = match[0];
        const entropy = calculateShannonEntropy(token);
        if (entropy >= threshold) {
            // Eliminate common false positives (e.g. repeated chars or CSS hashe
s)
            const uniqueChars = new Set(token).size;
            if (uniqueChars >= 8) {
                findings.push({
                    type: "HIGH_ENTROPY_SECRET",
                    value: token,
                    entropy: parseFloat(entropy.toFixed(3)),
                    startIndex: match.index,
                    endIndex: match.index + token.length,
                    severity: entropy >= 4.2 ? "CRITICAL" : "HIGH"
                });
            }
        }
    }
    return findings;
}
```

16.5 Known Secret Signature Matching

```
const SECRET_SIGNATURES = [
    { name: "AWS_ACCESS_KEY", regex: /\b(A3T[A-Z0-9]|AKIA|AGPA|AIDA|AROA|AIPA|ANPA
|ANVA|ASIA)[A-Z0-9]{16}\b/g, severity: "CRITICAL" },
    { name: "OPENAI_API_KEY", regex: /\bsk-(?:proj-|admin-|[a-zA-Z0-9]{20}T3BlbkF
J)[a-zA-Z0-9_\-]{20,100}\b/g, severity: "CRITICAL" },
    { name: "GITHUB_PAT", regex: /\b(?:ghp|gho|ghu|ghs|ghr)_[a-zA-Z0-9]{36,255}\b/
g, severity: "CRITICAL" },
    { name: "STRIPE_SECRET_KEY", regex: /\b(?:sk|rk)_(?:live|test)_[0-9a-zA-Z]{24,
99}\b/g, severity: "CRITICAL" },
    { name: "SLACK_BOT_TOKEN", regex: /\bxoxb-[0-9]{11,13}-[0-9]{11,13}-[a-zA-Z0-
9]{24}\b/g, severity: "CRITICAL" },
    { name: "GOOGLE_API_KEY", regex: /\bAIza[0-9A-Za-z_\-]{35}\b/g, severity: "CRI
TICAL" },
    { name: "DATABASE_CONNECTION_URI", regex: /\b(?:postgres|postgresql|mysql|mong
odb|redis):\/\/[^\s:]+:[^\s@]+@[^\s\/:]+(?::[0-9]+)?\/[^\s?#]+/g, severity: "CRITI
CAL" },
    { name: "RSA_PRIVATE_KEY", regex: /-----BEGIN (?:RSA )?PRIVATE KEY-----[\s\S]
*?-----END (?:RSA )?PRIVATE KEY-----/g, severity: "CRITICAL" }
];

function detectDeterministicSecrets(text) {
    const findings = [];
    SECRET_SIGNATURES.forEach(sig => {
        let match;
        sig.regex.lastIndex = 0;
        while ((match = sig.regex.exec(text)) !== null) {
            findings.push({
                type: sig.name,
                value: match[0],
                startIndex: match.index,
                endIndex: match.index + match[0].length,
                severity: sig.severity
            });
        }
    });
    return findings;
}
```

16.6 Luhn Algorithm Credit Card Verification

```
/**
 * Validates a candidate numeric sequence using the Luhn Modulo-10 checksum
 * @param {string} digits - Extracted candidate credit card digits
 * @returns {boolean} True if valid primary account number
 */
function validateLuhnChecksum(digits) {
    const clean = digits.replace(/[\s\-]/g, "");
    if (!/^[0-9]{13,19}$/.test(clean)) return false;
    let sum = 0;
    let shouldDouble = false;
    for (let i = clean.length - 1; i >= 0; i--) {
        let digit = parseInt(clean.charAt(i), 10);
        if (shouldDouble) {
            digit *= 2;
            if (digit > 9) digit -= 9;
        }
        sum += digit;
        shouldDouble = !shouldDouble;
    }
    return (sum % 10) === 0;
}
```

16.7 Personally Identifiable Information (PII) Scanner

```
function detectPII(text) {
    const findings = [];
    // US Social Security Number
    const ssnRegex = /\b(?!000|666|9\d{2})\d{3}-(?!00)\d{2}-(?!0000)\d{4}\b/g;
    let match;
    while ((match = ssnRegex.exec(text)) !== null) {
        findings.push({
            type: "PII_SSN",
            value: match[0],
            startIndex: match.index,
            endIndex: match.index + match[0].length,
            severity: "HIGH"
        });
    }
    // Credit Cards with Luhn Checksum
    const ccRegex = /\b(?:4[0-9]{12}(?:[0-9]{3})?|5[1-5][0-9]{14}|3[47][0-9]{13}|6
(?:011|5[0-9]{2})[0-9]{12})[\s\-]?[0-9]{4}[\s\-]?[0-9]{4}\b/g;
    while ((match = ccRegex.exec(text)) !== null) {
        if (validateLuhnChecksum(match[0])) {
            findings.push({
                type: "PII_CREDIT_CARD",
                value: match[0],
                startIndex: match.index,
                endIndex: match.index + match[0].length,
                severity: "HIGH"
            });
        }
    }
    return findings;
}
```

16.8 Reverse-Splice In-Place Auto-Redaction Engine

```
/**
 * Sanitizes prompt text by substituting detected tokens with semantic placeholder
s
 * Executes right-to-left to preserve substring index coordinates
 */
function redactPromptInPlace(originalText, findings) {
    if (!findings || findings.length === 0) return originalText;
    // Sort descending by startIndex
    const sorted = [...findings].sort((a, b) => b.startIndex - a.startIndex);
    let sanitized = originalText;
    let idx = sorted.length;
    for (const finding of sorted) {
        const placeholder = `[REDACTED_${finding.type}_${idx}]`;
        sanitized = sanitized.slice(0, finding.startIndex) + placeholder + sanitiz
ed.slice(finding.endIndex);
        idx--;
    }
    return sanitized;
}
```

16.9 Secret Masking Engine for Audit Logging

```
/**
 * Produces anchor-preserved masked preview for secure local auditing
 * e.g. "AKIAIOSFODNN7EXAMPLE" -> "AKIA••••••••••••MPLE"
 */
function maskSecretPreview(secret) {
    if (!secret || secret.length <= 8) return "••••••••";
    const prefix = secret.slice(0, 4);
    const suffix = secret.slice(-4);
    const middleMask = "•".repeat(Math.min(12, secret.length - 8));
    return `${prefix}${middleMask}${suffix}`;
}
```

16.10 Capture-Phase DOM Interceptor

```
// Register capture-phase listener to trap Enter key before React SyntheticEvents
window.addEventListener("keydown", function(e) {
    if (e.key === "Enter" && !e.shiftKey) {
        const target = e.target;
        if (target.matches("textarea, div[contenteditable='true']")) {
            const text = target.value || target.innerText;
            const evalResult = window.SentinelEngine.evaluate(text);
            if (evalResult.riskScore > 0) {
                e.preventDefault();
                e.stopImmediatePropagation();
                window.SentinelUI.showWarningModal(target, text, evalResult);
            }
        }
    }
}, true);
```

REFERENCES

1. C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, Jul. 1948.
2. H. P. Luhn, “Computer for Verifying Numbers,” U.S. Patent 2 950 048, Aug. 23, 1960.
3. OWASP Top 10 for Large Language Model Applications, “LLM06: Sensitive Information Disclosure,” *OWASP Foundation*, Version 1.1, Nov. 2023.
4. NIST Special Publication 800-53, Rev. 5, “Security and Privacy Controls for Information Systems and Organizations,” *National Institute of Standards and Technology*, Sep. 2020.
5. Google Chrome Extensions Documentation, “Manifest V3 Security and Architectural Overview,” *Chrome for Developers*, May 2024. [Online]. Available: https://developer.chrome.com/docs/extensions/mv3/intro/
6. M. M. Al-Rubaie and J. M. Chang, “Privacy-Preserving Machine Learning: Threats and Solutions,” *IEEE Security & Privacy*, vol. 17, no. 2, pp. 49–58, Mar. 2019.
7. N. Carlini et al., “Extracting Training Data from Large Language Models,” in *Proc. 30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2633–2650.
8. P. Henderson et al., “Foundation Models and the Law,” *IEEE Security & Privacy*, vol. 21, no. 3, pp. 42–51, May 2023.
9. S. Yao et al., “A Survey on Data Loss Prevention: Techniques, Implementations, and Challenges,” *IEEE Access*, vol. 8, pp. 194212–194231, Oct. 2020.
10. D. M. Blei, A. Y. Ng, and M. I. Jordan, “Latent Dirichlet Allocation,” *Journal of Machine Learning Research*, vol. 3, pp. 993–1022, Mar. 2003.
11. J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT 2019*, 2019, pp. 4171–4186.
12. A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS 30)*, 2017, pp. 5998–6008.
13. T. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems (NeurIPS 33)*, 2020, pp. 1877–1901.
14. Cloud Security Alliance (CSA), “Security Guidance for Critical Areas of Focus in Cloud Computing v4.0,” *Cloud Security Alliance*, Jul. 2017.
15. Payment Card Industry Security Standards Council, “Payment Card Industry (PCI) Data Security Standard Requirements and Testing Procedures Version 4.0,” Mar. 2022.
16. European Parliament and Council, “Regulation (EU) 2016/679 (General Data Protection Regulation),” *Official Journal of the European Union*, vol. L119, pp. 1–88, May 2016.

17. U.S. Department of Health and Human Services, “Health Insurance Portability and Accountability Act (HIPAA) Privacy Rule,” *Federal Register*, vol. 65, no. 250, Dec. 2000.

18. R. Fielding et al., “Hypertext Transfer Protocol -- HTTP/1.1,” *IETF RFC 2616*, Jun. 1999.

19. E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” *IETF RFC 8446*, Aug. 2018.

20. D. Dworkin et al., “Content Security Policy Level 3,” *W3C Working Draft*, Oct. 2021. [Online]. Available: https://www.w3.org/TR/CSP3/

21. J. R. Douceur, “The Sybil Attack,” in *Peer-to-Peer Systems (IPTPS 2001)*, Lecture Notes in Computer Science, vol. 2429, Springer, Berlin, Heidelberg, 2002, pp. 251–260.

22. B. Schneier, *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, 20th Anniversary Edition, John Wiley & Sons, 2015.

23. A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.

24. GitGuardian, “The State of Secrets Sprawl 2024: Analysis of Secrets Exposed on GitHub,” *GitGuardian Security Research*, Jan. 2024.

25. Truffle Security, “TruffleHog: High-Entropy and Regex Secret Search in Git Repositories,” 2024. [Online]. Available: https://github.com/trufflesecurity/trufflehog